PardoX

High-Performance DataFrame Engine

The Speed of Rust. The Simplicity of Python.

Version 0.3.2

- ✦ ~4.6 GB/s .prdx read throughput
- ✦ SIMD / AVX2 / NEON acceleration
- ✦ Zero-copy NumPy & Arrow bridge
- ✦ Python • Node.js • PHP • Java SDKs
- ✦ PostgreSQL • MySQL • SQL Server • MongoDB
- ✦ 181 C-ABI FFI exports across 5 Rust crates
- ✦ 29 feature gaps — fully implemented & validated

www.pardox.io | MIT License | 2026

Table of Contents

1. Overview
2. Getting Started — Installation
3. Getting Started — Quick Start
4. Getting Started — Roadmap
5. User Guide — Overview
6. User Guide — Input / Output
7. User Guide — Aggregations & Observer
8. User Guide — Data Mutation
9. User Guide — Databases
10. User Guide — GPU Acceleration
11. User Guide — ML Integration
12. SDK Documentation — Python
13. SDK Documentation — Node.js
14. SDK Documentation — PHP
15. SDK Documentation — Database Integration
16. FFI Exports Reference
17. Full API Reference
18. Base Knowledge — Validation Scripts Index

Overview

Welcome to PardoX

The Speed of Rust. The Simplicity of Python.
PardoX is a high-performance DataFrame engine designed for modern data engineering. It combines the safety and speed of a Rust Core with the ease of use of Python, PHP, and Node.js SDKs — allowing you to process massive datasets and integrate with any database without learning a new language.

Why PardoX?

Zero-Copy Architecture: Data flows directly from disk or database into memory-mapped Rust buffers — no Python objects, no intermediate copies.
SIMD Acceleration: Mathematical operations use AVX2/NEON CPU instructions for 5x–20x speedups vs. Python loops.
Universal Compatibility: Runs natively on Windows, Linux, and macOS (Intel & Apple Silicon).
Native Database Engine: Connect to PostgreSQL, MySQL, SQL Server, and MongoDB entirely through Rust — no Python database drivers required.
Multi-SDK: A single Rust core powers identical APIs in Python, Node.js, and PHP.
Native Format: The .prdx binary format enables ~4.6 GB/s read throughput for repeated workloads.

Documentation
Getting Started

Installation — Setup guide for Python, Node.js, and PHP.
Quick Start — Build your first ETL pipeline in 5 minutes.
Roadmap — What's coming in v0.4 and beyond.

User Guide

Input / Output — Multi-threaded CSV, native .prdx format, Apache Arrow bridge.
Databases — PostgreSQL, MySQL, SQL Server, MongoDB — read, write, and execute.
Data Mutation — Vectorized arithmetic, type casting, sorting, data cleaning.
Aggregations & Observer — Metrics, statistics, value counts, and full-DataFrame export.
GPU Acceleration — GPU Bitonic sort and CPU fallback.

ML Integration — Zero-copy NumPy bridge and Scikit-Learn compatibility.

⚙ API Reference

Full Reference — Detailed documentation of all classes, functions, and methods.
FFI Exports Reference — All 181 C-ABI functions exported by the Rust core across 5 crates. Use this to build custom bindings or validate SDK integrations.

Base Knowledge

Base Knowledge — Validation scripts for all 29 feature gaps across Python, Node.js, and PHP SDKs.
Python validation scripts — 28 files (validate_gap1_sdk.py → validate_gap29_sdk.py)
Node.js validation scripts — 18 files (validate_gap1_sdk.js → validate_gap18_sdk.js)
PHP validation scripts — 28 files (validate_gap1_sdk.php → validate_gap29_sdk.php)

SDK Documentation

SDK Documentation — In-depth guides for each SDK.
Python SDK — API reference and v0.3.2 features
Node.js SDK — API reference and v0.3.2 features
PHP SDK — API reference and v0.3.2 features
Database Integration — PostgreSQL, MySQL, SQL Server, MongoDB across all SDKs
Universality — Cross-SDK design philosophy

Examples & Notebooks

Jupyter Notebooks — Interactive examples and real-world ETL scenarios.
Benchmark Scripts — Processing 640 million rows and transforming data to .prdx format.

Quick Install
pip install pardox

What's New in v0.3.2

Pillar

What was added

PRDX Streaming to PostgreSQL

write_sql_prdx() — stream any .prdx file directly to PostgreSQL via COPY FROM STDIN with O(block) RAM. Validated: 150M rows / 3.8 GB in ~490s at ~300k rows/s (Python/JS)

Gaps 1–5 — All SDKs

GroupBy, String & Date ops, Decimal type, Window functions, Lazy pipeline — validated across Python, JavaScript, and PHP SDKs

Gaps 7–14 — Python

GPU compute, Pivot & Melt, Time Series Fill, Nested Data (JSON), Spill to Disk, Universal Loader (PRDX), SQL over DataFrames

Gaps 15–29 — Python

Cloud Storage, Live Query, WebAssembly, Encryption, Data Contracts, Time Travel, Arrow Flight, Distributed Cluster, Linear Algebra, REST Connector

VAP31 & VAP32

CSV→PostgreSQL and PRDX→PostgreSQL integrations validated in 3 SDKs

29 Gaps Total

All 29 feature gaps from the original roadmap implemented in the Rust core

FFI Reference

Complete documentation of all 181 C-ABI exports across 5 crates

What's New in v0.3.1

Pillar
What was added

Relational Conqueror
Native read/write/execute for PostgreSQL, MySQL, SQL Server, MongoDB via Rust drivers

The Observer
to_dict(), to_json(), value_counts(), unique() — full-DataFrame export with proper heap memory management

Native Math
df.add(), df.sub(), df.std(), df.min_max_scale(), df.sort_values() — pure Rust arithmetic

GPU Awakening
sort_values(gpu=True) — WebGPU Bitonic sort with automatic CPU fallback

ML Integration
Zero-copy NumPy bridge via __array__ protocol — direct pointer into Rust buffer

PHP & Node.js SDKs
Full parity with Python SDK across all new features

Open Source Project distributed under the MIT License.
More info: www.pardox.io

Getting Started — Installation

Installation

PardoX ships as a pre-compiled native library. No Rust compiler or C toolchain is required on the end-user machine. The Rust core (libpardox.so / .dylib / .dll) is bundled inside the package for each supported platform.

Python SDK

via pip

pip install pardox

System Requirements

OS	Architecture	Status
Linux	x86_64	Stable
Windows	x86_64	Stable
macOS	ARM64 (M1/M2/M3)	Stable
macOS	x86_64 (Intel)	Stable

Note: Python 3.8 or higher is required. No Rust compiler is needed — binaries are pre-packaged for all platforms.
Note: PardoX connects to PostgreSQL, MySQL, SQL Server, and MongoDB entirely through the Rust core. You do not need to install psycopg2, pymysql, pyodbc, or pymongo.

Verify Installation

```
import pardox as px
print(px.__version__)    # e.g. "0.3.2"

df = px.read_csv("my_data.csv")
print(df.shape)
```

Node.js SDK

The Node.js SDK uses koffi for FFI bindings and ships the same pre-compiled .so / .dylib / .dll.

```
npm i @pardox/pardox
```

Usage

```
const { DataFrame, read_csv } = require('@pardox/pardox');

const df = px.read_csv('my_data.csv');
console.log(df.shape);
```

Requirements

Runtime

Minimum version

Node.js

18 LTS or higher

PHP SDK

The PHP SDK uses the built-in FFI extension (PHP 7.4+).

composer require betoalien/pardox-php

Requirements

Runtime

Minimum version

PHP

8.1 or higher

FFI extension

Must be enabled in php.ini

Enable the FFI extension:

```
; php.ini
extension=ffi
ffi.enable=true
```

Usage

```
shape[0] . " rows\n";
```

Docker (Databases for Testing)

A docker-compose.yml is provided to spin up all supported databases locally:
docker-compose up -d

This starts:

Container
Database
Host port

pardox_validation_db
PostgreSQL 16
5434

pardox_mysql
MySQL 8.0
3306

pardox_sqlserver
SQL Server 2022
1433

Tip: Add a MongoDB service to the compose file with mongo:7.0 if needed for your environment.

Getting Started — Quick Start

Quick Start

This guide walks through a complete v0.3.2 data pipeline: loading a CSV, running vectorized math, inspecting results, and writing to a relational database — all in native Rust speed.

Hello World

```
import pardox as px

# 1. Load data — parallel Rust CSV parser, automatic type inference
df = px.read_csv("sales_data.csv")
print(f"Loaded {df.shape[0]:,} rows × {df.shape[1]} columns")

# 2. Inspect columns
print(df.columns)
df.show(5)   # ASCII table preview

# 3. Cast and compute
df.cast("quantity", "Float64")
revenue_df = df.mul("price", "quantity")   # new DataFrame with 'result_mul'

# 4. Statistics
std_val = revenue_df.std("result_mul")
print(f"Revenue std dev: {std_val:,.2f}")

# 5. Write to PostgreSQL (COPY FROM STDIN auto-activated for > 10k rows)
from pardox.io import execute_sql
CONN = "postgresql://user:password@localhost:5432/mydb"
execute_sql(CONN, "CREATE TABLE IF NOT EXISTS sales (price FLOAT, quantity FLOAT)")
rows = df.to_sql(CONN, "sales", mode="append")
print(f"Written {rows:,} rows to PostgreSQL")
```

Step-by-Step Breakdown

1. Ingestion (read_csv)

PardoX spawns a Rust thread pool to parse chunks of the CSV file in parallel. No Python objects are created during ingestion — data flows directly into Rust-managed memory buffers.

```
df = px.read_csv("dataset.csv")
```

Tip: The engine scans the first rows to classify each column as Int64, Float64, or Utf8 (string). You can override with an explicit schema:

```python
df = px.read_csv("data.csv", schema={"price": "Float64", "id": "Int64"})
```

2. Inspect (show, shape, columns, dtypes)

```
print(df.shape)      # (50000, 14)
print(df.columns)    # ['id', 'price', 'quantity', ...]
print(df.dtypes)     # {'id': 'Utf8', 'price': 'Float64', ...}
df.show(5)           # ASCII table, first 5 rows
```

3. Type Casting (cast)

When a column is inferred as Int64 but you need Float64 for arithmetic:

```
df.cast("quantity", "Float64")
```

Supported types: Int64, Float64, Utf8.

4. Vectorized Arithmetic

```
# Series operators (via Proxy / __getitem__)
df['total'] = df['price'] * df['quantity']
df['tax']   = df['total'] * 0.16

# DataFrame-level methods (return new DataFrame)
revenue_df    = df.mul("price", "quantity")   # result column: 'result_mul'
profit_df     = df.sub("revenue", "cost")     # result column: 'result_sub'
```

5. Native Math Methods

```
# Standard deviation (pure Rust, no NumPy)
std_val = revenue_df.std("result_mul")

# Min-Max normalization → new DataFrame with 'result_minmax' column
```

```
normed_df = df.min_max_scale("price")

# Sort by column (CPU or GPU)
sorted_df = df.sort_values("price", ascending=False)
sorted_df = df.sort_values("price", ascending=True, gpu=True)  # GPU Bitonic sort
```

6. Observer — Data Inspection & Export

```
# Value frequency table
state_counts = df.value_counts("state")
print(state_counts)   # {'TX': 6345, 'CA': 6301, ...}

# Unique values in a column
unique_cats = df.unique("category")

# Export full DataFrame to Python
records = df.to_dict()   # list of dicts — 50k records
json_str = df.to_json()  # JSON string "[{...}, ...]"
```

7. Database Write (Relational Conqueror)

```
from pardox.io import execute_sql, execute_mysql, read_sql

PG_CONN    = "postgresql://pardox:secret@localhost:5432/mydb"
MYSQL_CONN = "mysql://pardox:secret@localhost:3306/mydb"

# PostgreSQL — auto COPY FROM STDIN for > 10k rows
execute_sql(PG_CONN, "CREATE TABLE IF NOT EXISTS sales (price FLOAT, quantity FLOAT)")
rows = df.to_sql(PG_CONN, "sales", mode="append")
print(f"Postgres: {rows:,} rows written")

# MySQL — chunked batch INSERT (auto LOAD DATA if server allows)
execute_mysql(MYSQL_CONN, "CREATE TABLE IF NOT EXISTS sales (price DOUBLE, quantity
DOUBLE)")
rows = df.to_mysql(MYSQL_CONN, "sales", mode="append")
print(f"MySQL: {rows:,} rows written")

# Read back from PostgreSQL
df_check = read_sql(PG_CONN, "SELECT COUNT(*) FROM sales")
```

8. Zero-Copy NumPy Integration

```
import numpy as np

# Direct pointer from Rust buffer — no data copy
arr = np.array(df["price"])
print(arr.dtype)   # float64
print(arr.mean())  # same value as df["price"].mean()
```

9. Persist to .prdx

```
# Save
df.to_prdx("sales_processed.prdx")

# Load (4.6 GB/s read throughput)
df2 = px.read_prdx("sales_processed.prdx")
```

Example: Loading 2 GB of data: CSV ~8s · Parquet ~3s · PRDX ~0.5s

Full Pipeline Example

```
import pardox as px
from pardox.io import execute_sql

CONN = "postgresql://pardox:secret@localhost:5432/analytics"

# Load
df = px.read_csv("sales_50k.csv")
df.cast("quantity", "Float64")

# Transform
df.fillna(0.0)
df['revenue'] = df['price'] * df['quantity']
df['tax']     = df['revenue'] * 0.08

# Analyze
```

```
print(f"Total revenue : ${df['revenue'].sum():,.2f}")
print(f"Avg ticket    : ${df['revenue'].mean():,.2f}")
print(f"Std deviation : {df['revenue'].std():,.2f}")

# Inspect
top_states = df.value_counts("state")
print("Top states:", list(top_states.items())[:5])

# Write
execute_sql(CONN, "DROP TABLE IF EXISTS sales_results")
execute_sql(CONN, (
    "CREATE TABLE sales_results "
    "(price FLOAT, quantity FLOAT, revenue FLOAT, tax FLOAT)"
))
rows = df.to_sql(CONN, "sales_results", mode="append")
print(f"\nWrote {rows:,} rows to PostgreSQL")

# Save locally
df.to_prdx("sales_results.prdx")
print("Saved to sales_results.prdx")
```

Getting Started — Roadmap

Roadmap

This page tracks features planned for upcoming releases and the reasoning behind them.

v0.3.1 — Released (2026-02-28)

All items below were delivered in this release.

Feature	Status
PostgreSQL read / write / execute	Done
MySQL read / write / execute	Done
SQL Server read / write / execute	Done
MongoDB read / write / execute	Done
PostgreSQL COPY FROM STDIN (> 10k rows)	Done
MySQL LOAD DATA LOCAL INFILE (> 10k rows)	Done (server-side flag required)
SQL Server batch INSERT 500 rows/stmt	

Done

SQL Server MERGE upsert
Done

MongoDB ordered:false, 10k doc batches
Done

to_dict(), to_json() — full DataFrame export
Done

value_counts(), unique() — EDA Observer
Done

df.add(), df.sub(), df.std(), df.min_max_scale()
Done

df.sort_values(ascending, gpu)
Done

GPU Bitonic sort (WebGPU / wgpu) with CPU fallback
Done

Zero-copy NumPy bridge (__array__ protocol)
Done

PHP SDK — full feature parity
Done

Node.js SDK — full feature parity
Done

FFI memory safety — heap vs. thread-local string ownership
Fixed

v0.3.2 — Released (2026-03-15)
All items below were delivered in this release.

Feature
Status

write_sql_prdx() — PRDX streaming to PostgreSQL via COPY FROM STDIN, O(block) RAM
Done

GroupBy aggregation (Gap 1)
Done — Python, JS, PHP

String & Date operations (Gap 2)
Done — Python, JS, PHP

Decimal type (Gap 3)
Done — Python, JS, PHP

Window functions (Gap 4)
Done — Python, JS, PHP

Lazy pipeline (Gap 5)
Done — Python, JS, PHP

GPU compute (Gap 7)
Done — Python, PHP

Pivot & Melt (Gap 8)
Done — Python, PHP

Time Series Fill (Gap 9)
Done — Python, PHP

Nested Data / JSON (Gap 10)
Done — Python, PHP

Spill to Disk (Gap 11)
Done — Python, PHP

Universal Loader PRDX (Gap 12)
Done — Python, PHP

Streaming GroupBy (Gap 13)
Done — PHP

SQL over DataFrames (Gap 14)
Done — Python, JS, PHP

Cloud Storage (Gap 15)
Done — Python

Live Query (Gap 16)
Done — Python

WebAssembly (Gap 17)
Done — JS

Encryption (Gap 18)
Done — Python

Data Contracts (Gap 19)
Done — Python

Time Travel (Gap 20)
Done — Python

Arrow Flight (Gap 21)
Done — Python

Distributed Cluster (Gap 22)
Done — Python

SQL Server Fix (Gap 23)
Done — Python

Fault Tolerance (Gap 26)
Done — Python

Query Planner (Gap 27)
Done — Python

Linear Algebra (Gap 28)
Done — Python

REST Connector (Gap 29)
Done — Python

FFI Reference documentation (181 exports)
Done

VAP31 CSV→PostgreSQL — 3 SDKs
Done

VAP32 PRDX→PostgreSQL — 3 SDKs
Done — 150M rows validated

v0.4.0 — Planned
SQL Server — Password Special Character Fix
Problem: The tiberius Rust driver (v0.12) fails to authenticate when the SQL Server password contains ! or certain other special characters.
Planned fix: Audit tiberius v0.12.x login.rs and patch or upgrade to a version where the encoding is correct for all printable ASCII characters.
Workaround until fixed: Use a SQL Server password that contains only [A-Za-z0-9_\-@#$].

Error Code Hierarchy
Problem: All database errors currently return -100 regardless of the actual failure reason, making diagnostics difficult.
Planned improvement:

Code range
Category

-100 to -199
Connection / authentication errors

-200 to -299
Query parse / syntax errors

-300 to -399
Data type mismatch

-400 to -499
Constraint violations

-500 to -599
Timeout errors

Additional Native Data Sources

Source
Format / Protocol
Notes

Apache Parquet
Binary columnar
Direct read into HyperBlock via parquet2 crate

Apache Kafka
Streaming
Topic → DataFrame streaming ingestion

AWS S3
Object storage
CSV / Parquet over S3 via aws-sdk-rust

Snowflake
JDBC-like
Via Snowflake Arrow export

Oracle DB
OCI
Via oracle Rust crate

SQLite
File-based SQL
Via rusqlite

Gaps 7–14 — JavaScript SDK
JavaScript SDK currently segfaults on Gaps 7–11 due to a known Core issue. Fix and re-validate planned.

Long-Term Vision

Distributed execution: Partition HyperBlocks across nodes via a Rust-native scheduler.
Streaming DataFrames: Ingest from Kafka / Kinesis as append-only HyperBlock partitions.
GPU-accelerated aggregations: Extend GPU support from sort to sum, mean, and groupby using compute shaders.
WASM SDK: Full PardoX in the browser via WebAssembly.

User Guide — Overview

User Guide

Explore core workflows, data operations, and advanced acceleration features.

Input / Output Operations
Database Integration
Data Mutation & Arithmetic
Aggregations, Analytics & The Observer
GPU Acceleration
ML Integration — Zero-Copy NumPy Bridge

User Guide — Input / Output

Input / Output Operations

The bottleneck in most data pipelines is not computation, but IO. PardoX moves all data ingestion and persistence entirely into the Rust core, bypassing Python's file handling and object creation overhead.

1. CSV Files

PardoX features a multi-threaded CSV reader. The file is memory-mapped and parallel workers parse chunks simultaneously. No Python objects are created during ingestion.

Basic usage

```
import pardox as px

# Automatic type inference (Int64, Float64, Utf8)
df = px.read_csv("dataset.csv")
print(df.shape)    # (rows, cols)
print(df.columns)  # ['col1', 'col2', ...]
```

Manual schema

```
df = px.read_csv("dataset.csv", schema={
    "price":    "Float64",
    "quantity": "Float64",
    "id":       "Int64",
    "name":     "Utf8",
})
```

Info: Without a schema, the engine scans the first rows to detect Int64, Float64, or Utf8. Pass a schema to override for specific columns.

Success: The file is split into logical blocks processed by multiple CPU cores concurrently.

2. Native Format (.prdx)

The PRDX format is a custom binary layout designed for instant persistence. Reading is a near-direct memory copy — no parsing, no schema detection.

Save

```
df.to_prdx("output.prdx")
```

Load

```
df = px.read_prdx("output.prdx")
```

Success: In tests with 10 GB datasets, reading a .prdx file achieves ~4.6 GB/s throughput, limited only by NVMe SSD speed.

Format

Read speed (2 GB file)

CSV

~8s

Parquet

~3s

PRDX

~0.5s

3. Apache Arrow Bridge

PardoX integrates with the Arrow ecosystem via the Arrow C Data Interface. Conversion passes memory pointers — no data is copied.

```
import pyarrow as pa
import pardox as px

arrow_table = pa.Table.from_pydict({"price": [1.5, 2.0], "qty": [10, 20]})
df = px.from_arrow(arrow_table)
```

Tip: The Arrow bridge connects PardoX to Polars, DuckDB, Apache Spark, and any other tool that supports the Arrow IPC format.

4. PostgreSQL

PardoX uses tokio-postgres inside the Rust core. The host language (Python / Node.js / PHP) never touches the database wire protocol.

Connection string

postgresql://user:password@host:port/database

Read

```
from pardox.io import read_sql

df = read_sql(
    "postgresql://pardox:secret@localhost:5432/mydb",
    "SELECT id, amount, region FROM orders WHERE status = 'complete'"
)
```

Execute DDL / DML

```
from pardox.io import execute_sql

execute_sql(CONN, "DROP TABLE IF EXISTS orders_archive")
execute_sql(CONN, """
    CREATE TABLE orders_archive (
        id      BIGINT,
        amount  DOUBLE PRECISION,
        region  TEXT
    )
""")
# Returns 0 for DDL, affected rows for DML
n = execute_sql(CONN, "DELETE FROM orders WHERE status = 'cancelled'")
```

Write

```
# mode='append' — auto-activates COPY FROM STDIN for > 10,000 rows
rows = df.to_sql(CONN, "orders_archive", mode="append")
print(f"{rows:,} rows written")
```

```
# mode='upsert' — INSERT … ON CONFLICT (cols) DO UPDATE SET …
rows = df.to_sql(CONN, "orders_archive", mode="upsert", conflict_cols=["id"])
```

Success: When writing more than 10,000 rows with mode="append", the Rust core automatically switches to PostgreSQL's COPY FROM STDIN protocol. The CSV payload is serialized in memory and streamed at wire speed — no temp files. This is typically 10x–50x faster than multi-row INSERT.

5. MySQL

PardoX uses the mysql crate v25.

Connection string

mysql://user:password@host:port/database

Read

```
from pardox.io import read_mysql

df = read_mysql(
    "mysql://pardox:secret@localhost:3306/mydb",
    "SELECT * FROM products WHERE active = 1"
)
```

Execute DDL / DML

```
from pardox.io import execute_mysql

execute_mysql(CONN, "DROP TABLE IF EXISTS products_bak")
execute_mysql(CONN, """
    CREATE TABLE products_bak (
        id       BIGINT,
        name     TEXT,
        price    DOUBLE,
        quantity DOUBLE
    )
""")
```

Write

```
# append — chunked INSERT 1,000 rows/stmt (auto LOAD DATA for > 10k if server allows)
rows = df.to_mysql(CONN, "products_bak", mode="append")

# replace — REPLACE INTO (delete + insert)
rows = df.to_mysql(CONN, "products_bak", mode="replace")

# upsert — INSERT … ON DUPLICATE KEY UPDATE
rows = df.to_mysql(CONN, "products_bak", mode="upsert", conflict_cols=["id"])
```

Info: For append with more than 10,000 rows, PardoX attempts LOAD DATA LOCAL INFILE. If the server has local_infile=OFF (MySQL default), it falls back automatically to the 1,000-row chunked INSERT and logs a notice to stderr. To enable the fast path:

```sql
SET GLOBAL local_infile = 1;
```

6. SQL Server

PardoX uses the tiberius crate v0.12 with a single-thread Tokio runtime.

Connection string (ADO.NET format)

Server=host,port;Database=db;UID=user;PWD=password;TrustServerCertificate=Yes

Read

```
from pardox.io import read_sqlserver

CONN = "Server=localhost,1433;Database=mydb;UID=sa;PWD=MyPassword;TrustServerCertificate=Yes"

df = read_sqlserver(CONN, "SELECT TOP 1000 * FROM dbo.orders")
```

Execute DDL / DML

```
from pardox.io import execute_sqlserver

execute_sqlserver(CONN, "DROP TABLE IF EXISTS dbo.orders_bak")
```

```
execute_sqlserver(CONN, """
    CREATE TABLE dbo.orders_bak (
        id       BIGINT,
        amount   FLOAT,
        region   NVARCHAR(MAX)
    )
""")
```

Write

```
# append — multi-row INSERT, 500 rows per statement
rows = df.to_sqlserver(CONN, "dbo.orders_bak", mode="append")

# upsert — MERGE INTO … WHEN MATCHED … WHEN NOT MATCHED
rows = df.to_sqlserver(CONN, "dbo.orders_bak", mode="upsert", conflict_cols=["id"])
```

Warning: Avoid using ! in SQL Server passwords. A known issue in tiberius v0.12 causes authentication failure when ! is present in the password when connecting via TCP from an external host. Use only [A-Za-z0-9_\-@#$]. A fix is tracked for v0.3.2.
Info: SQL Server writes use 500-row multi-value INSERT statements. For 50,000 rows this results in 100 round-trips instead of 50,000. A BULK INSERT / bcp path is planned for v0.3.2.

7. MongoDB

PardoX uses the mongodb crate v2.8.
Connection string (MongoDB URI)
mongodb://user:password@host:port

Read

```
from pardox.io import read_mongodb

# Target format: "database.collection"
df = read_mongodb("mongodb://admin:secret@localhost:27017", "mydb.orders")
```

Execute commands

```
from pardox.io import execute_mongodb
```

```
# Drop a collection
execute_mongodb("mongodb://...", "mydb", '{"drop": "orders_bak"}')
```

Write

```
# append — insert_many in batches of 10,000 documents, ordered:false
rows = df.to_mongodb("mongodb://...", "mydb.orders_bak", mode="append")

# replace — drops collection, then inserts all documents
rows = df.to_mongodb("mongodb://...", "mydb.orders_bak", mode="replace")
```

Success: ordered: false is used on every insert_many call. This means MongoDB continues inserting valid documents even if individual documents fail (e.g., duplicate key), rather than aborting the entire batch.

Write Modes Summary

Database
append
replace
upsert

PostgreSQL
INSERT (COPY for >10k)
—
INSERT ON CONFLICT DO UPDATE

MySQL
INSERT 1k/stmt (LOAD DATA for >10k)
REPLACE INTO
INSERT ON DUPLICATE KEY UPDATE

SQL Server
Multi-row INSERT 500/stmt
Multi-row INSERT 500/stmt
MERGE INTO

MongoDB
insert_many 10k/batch
drop + insert_many
—

Table Must Exist Before Writing
PardoX write methods append to an existing table and do not auto-create it. Always call execute_* first:

```
from pardox.io import execute_sql

execute_sql(CONN, """
    CREATE TABLE IF NOT EXISTS my_table (
        id      BIGINT,
        amount  DOUBLE PRECISION,
        label   TEXT
    )
""")

rows = df.to_sql(CONN, "my_table", mode="append")
```

User Guide — Aggregations & Observer

Aggregations, Analytics & The Observer
PardoX provides two layers of data insight:

Aggregation kernels — reduce a column to a scalar (sum, mean, std, etc.)
The Observer — export and inspect the full DataFrame (value_counts, unique, to_dict, to_json)

All operations run entirely in native Rust machine code.

1. Column Aggregations (Series)
Access aggregation methods directly on any numeric Series object (obtained via df['col']).
sum()
```
total_revenue = df['amount'].sum()
print(f"Total: ${total_revenue:,.2f}")
```

Returns: float
mean()
```
avg_ticket = df['amount'].mean()
```

Returns: float
min() / max()
```
highest = df['amount'].max()
lowest  = df['amount'].min()
```

Returns: float
std()
Sample standard deviation.
```
volatility = df['amount'].std()
```

Returns: float

Info: Low std → values cluster near the mean. High std → values are widely spread.

count()
Number of non-null values in the column.
```
valid_transactions = df['transaction_id'].count()
```

Returns: int

2. DataFrame-Level Standard Deviation

df.std(col) is a DataFrame method (distinct from the Series .std()) that computes the sample standard deviation for a named column and returns a Python float. Useful when working with derived DataFrames from df.mul(), df.add(), etc.

```
revenue_df = df.mul("price", "quantity")
std_val    = revenue_df.std("result_mul")
print(f"Revenue std dev: {std_val:,.4f}")
```

3. Null Value Handling

All aggregation functions are null-aware: they skip NaN / null values and compute metrics only over valid data points. This matches SQL and Pandas behavior.

To fill nulls before aggregation:

```
df.fillna(0.0)
total = df['amount'].sum()
```

4. Performance

Method	Mechanism	Speed (10M rows)
Python sum(list)	Iterates PyObject one by one	~1.5s
PardoX .sum()	SIMD vectorized accumulator	

~0.02s

Tip: .sum() passes the memory pointer directly to a Rust function. AVX2 instructions add 4–8 values per CPU cycle without ever materializing Python objects.

5. The Observer — Full DataFrame Export

The Observer module provides functions to export or inspect the entire DataFrame. All string results are heap-allocated (proper ownership) and freed after the Python string is created.

to_dict()

Returns all rows as a list of dictionaries (records format). Equivalent to Pandas' df.to_dict('records').

```
records = df.to_dict()
# [{'price': 19.99, 'quantity': 3, ...}, ...]

print(f"Total records: {len(records)}")
first_row = records[0]
print(first_row['price'])
```

Returns: list[dict]

to_json()

Returns all rows as a JSON string "[{...}, ...]". Useful for API responses or file writing.

```
json_str = df.to_json()
with open("output.json", "w") as f:
    f.write(json_str)
```

Returns: str

value_counts(col)

Returns the frequency of each unique value in a column, sorted by count descending.

```
state_counts = df.value_counts("state")
# {'TX': 6345, 'CA': 6301, 'CO': 6304, ...}

print(f"Unique states: {len(state_counts)}")

# Top 5
for state, count in list(state_counts.items())[:5]:
```

```
    print(f"  {state}: {count:,}")
```

Returns: dict[str, int]
unique(col)
Returns the unique values of a column in insertion order.

```
categories = df.unique("category")
# ['Electronics', 'Books', 'Clothing', ...]

print(f"Distinct categories: {len(categories)}")
```

Returns: list

6. Observer — PHP and Node.js
Node.js

```
// value_counts
const stateCounts = df.valueCounts('state');
console.log(`States: ${Object.keys(stateCounts).length}`);

// unique
const cats = df.unique('category');

// Full export
const records = df.toDict();       // array of objects
const jsonStr = df.toJson();       // JSON string

// tolist — array of arrays (values only)
const matrix = df.tolist();
```

PHP

```
// value_counts
$stateCounts = $df->value_counts('state');
echo count($stateCounts) . " unique states\n";

// unique
$cats = $df->unique('category');

// Full export
$records = $df->to_dict();    // array of assoc arrays
```

```
$json    = $df->to_json();    // JSON string

// tolist — array of arrays
$matrix  = $df->tolist();
```

7. Complete Analysis Pipeline

```
import pardox as px
from pardox.io import execute_sql

df = px.read_csv("sales_50k.csv")
df.cast("quantity", "Float64")

# Feature engineering
revenue_df = df.mul("price", "quantity")

# Aggregations
print(f"Total revenue : ${df['price'].sum():,.2f}")
print(f"Avg price     : ${df['price'].mean():,.4f}")
print(f"Max price     : ${df['price'].max():,.2f}")
print(f"Revenue std   : {revenue_df.std('result_mul'):,.2f}")
print(f"Valid rows    : {df['transaction_id'].count():,}")

# EDA inspection
state_counts  = df.value_counts("state")
categories    = df.unique("category")
print(f"\nTop 3 states: {list(state_counts.items())[:3]}")
print(f"Categories:   {categories}")

# Full export
records = df.to_dict()
print(f"\nExported {len(records):,} records to Python list")
```

8. GroupBy (Roadmap)

Vectorized groupby with a Rust hash-aggregation engine is planned for v0.3.2.

```
# Coming in v0.3.2
summary = df.groupby("region").agg({
    "revenue": "sum",
```

```
    "quantity": "mean",
    "transaction_id": "count"
})
```

For now, use value_counts for frequency analysis and filter + aggregate on subsets:

```
# Workaround: filter then aggregate
tx_mask = df['state'].eq("TX")
tx_df   = df.filter(tx_mask)
print(f"TX revenue: ${tx_df['amount'].sum():,.2f}")
```

User Guide — Data Mutation

Data Mutation & Arithmetic

PardoX performs all arithmetic and transformations directly on Rust memory buffers using SIMD (Single Instruction, Multiple Data) instructions. No Python objects are created during computation.

1. Column Selection

```
# Select a column — returns a Series (lightweight Rust column pointer)
prices = df['price']
print(type(prices))  #
```

Info: Column selection returns a view into the Rust buffer, not a copy.

2. Vectorized Arithmetic (Series operators)

Operations via Python operators (+, -, *, /) are dispatched to SIMD-accelerated Rust kernels.

```
# Vector × Vector
df['total']      = df['price'] * df['quantity']

# Vector × Scalar (broadcast)
df['tax']        = df['total'] * 0.08

# Chained expressions
df['net_amount'] = (df['total'] + df['tax']) - df['discount']
```

Tip: AVX2 (Intel/AMD) and NEON (Apple Silicon) process 4–8 values per CPU cycle — 5x–20x faster than Python loops.

3. DataFrame Arithmetic Methods

These methods accept column names as strings and return a new DataFrame containing the result column.

df.add(col_a, col_b) — Addition

```
# Returns new DataFrame with column 'result_add'
sum_df = df.add("price", "tax")
print(sum_df.columns)   # ['result_add']
```

```
total = sum_df['result_add'].sum()
```

df.sub(col_a, col_b) — Subtraction

```
# Returns new DataFrame with column 'result_sub'
profit_df = df.sub("revenue", "cost")
```

df.mul(col_a, col_b) — Multiplication

```
# Returns new DataFrame with column 'result_mul'
revenue_df = df.mul("price", "quantity")
print(f"Revenue std dev: {revenue_df.std('result_mul'):,.2f}")
```

df.std(col) — Standard Deviation

Returns the sample standard deviation of a column as a scalar float. Pure Rust, no NumPy dependency.

```
std_val = df.std("amount")
print(f"Volatility: {std_val:.4f}")
```

df.min_max_scale(col) — Min-Max Normalization

Normalizes a column to the range [0, 1]. Returns a new DataFrame with column result_minmax.

```
# Normalize prices to [0, 1]
normed_df = df.min_max_scale("price")
print(normed_df['result_minmax'].min())   # ~0.0
print(normed_df['result_minmax'].max())   # ~1.0
```

4. Type Casting

Convert a column to a different type in-place.

```
# Required before arithmetic when column was inferred as Int64
df.cast("quantity", "Float64")
df.cast("id",       "Utf8")     # convert numeric ID to string
```

Supported types: Int64, Float64, Utf8

5. Sorting (sort_values)

Sort the DataFrame by a column. Returns a new sorted DataFrame.

```
# Ascending sort (CPU, Rust parallel merge sort)
sorted_df = df.sort_values("price", ascending=True)

# Descending sort
sorted_df = df.sort_values("price", ascending=False)

# GPU Bitonic sort (falls back to CPU if GPU unavailable)
sorted_df = df.sort_values("price", ascending=True, gpu=True)

print(f"Sorted {sorted_df.shape[0]:,} rows")
```

Info: If a GPU is not available or the wgpu backend cannot initialize, PardoX automatically uses the CPU sort and logs [PardoX GPU Sort] GPU not available, using CPU sort. to stderr. The result is identical.

See GPU Acceleration for details on the Bitonic sort pipeline.

6. Filtering

Apply a boolean Series as a row filter.

```
# Single condition
mask     = df['price'] > 100.0
filtered = df.filter(mask)

# Combined conditions
mask2    = df['state'].eq("TX")
result   = df.filter(mask).filter(mask2)
```

Comparison operators on Series:

Method

Operator

s.eq(val)
==

s.neq(val)
!=

s.gt(val)
>

s.gte(val)
>=

s.lt(val)
<

s.lte(val)
<=

7. Data Cleaning
fillna(value)
Fills NaN / null values in all numeric columns in-place.
df.fillna(0.0) # replaces nulls with 0 — modifies buffer directly

Warning: fillna modifies the DataFrame in-place. No need to reassign: df = df.fillna(0.0).

round(decimals)
Rounds all floating-point columns to N decimal places in-place.
df.round(2) # round all numerics to 2 decimal places

8. Slicing (iloc)

Select a range of rows by position.

```
# Rows 100 to 199 (start inclusive, end exclusive)
subset = df.iloc(100, 200)
print(subset.shape)   # (100, n_cols)
```

9. Joins

Hash-join two DataFrames on a key column.

```
# Inner join on the same key column name
result = orders.join(customers, on="customer_id")

# Join on different column names
result = orders.join(customers, left_on="cust_id", right_on="id")
```

10. Performance Best Practices

Success: - Use column arithmetic (df['c'] = df['a'] * df['b']) for all row-level computations.
- Cast columns to Float64 before arithmetic when they were inferred as Int64.
- Chain operations — Python handles precedence, Rust handles execution.
Danger: - Python loops over rows (for row in df) destroy performance. Always use column operations.

```
#  100x-1000x SLOWER — Python loop
for i in range(len(df)):
    df['new'][i] = df['a'][i] + df['b'][i]

#  FAST — SIMD vectorized
df['new'] = df['a'] + df['b']
```

User Guide — Databases

Database Integration

PardoX v0.3.2 provides native database connectivity via Rust drivers — no Python, PHP, or Node.js database libraries required. The same Rust core powers identical APIs across all three SDKs.

Architecture

```
Python / Node.js / PHP
      │
      │  connection string + DataFrame pointer
      ▼
  Rust Core (libpardox.so)
      │
      ├── tokio-postgres  → PostgreSQL
      ├── mysql v25       → MySQL / MariaDB
      ├── tiberius v0.12  → SQL Server
      └── mongodb v2.8    → MongoDB
```

The host language sends a connection string and a raw pointer to the HyperBlock. Rust manages the entire wire protocol, serialization, and bulk transfer. No ORM, no query builder, no intermediate Python objects.

Supported Databases

Database

Rust driver

Read

Write

Execute DDL/DML

PostgreSQL

tokio-postgres

MySQL / MariaDB
mysql v25

SQL Server
tiberius v0.12

MongoDB
mongodb v2.8

Connection String Formats

Database
Format

PostgreSQL
postgresql://user:password@host:port/database

MySQL

mysql://user:password@host:port/database

SQL Server
Server=host,port;Database=db;UID=user;PWD=password;TrustServerCertificate=Yes

MongoDB
mongodb://user:password@host:port

PostgreSQL
Python

```
from pardox.io import read_sql, execute_sql
import pardox as px

CONN = "postgresql://pardox:secret@localhost:5432/mydb"

# Read
df = read_sql(CONN, "SELECT * FROM orders WHERE status = 'active'")
print(df.shape)   # (rows, cols)

# DDL
execute_sql(CONN, "DROP TABLE IF EXISTS orders_processed")
execute_sql(CONN, """
    CREATE TABLE orders_processed (
        id      BIGINT,
        amount  DOUBLE PRECISION,
        region  TEXT,
        UNIQUE (id)
    )
""")

# Write — append (auto COPY FROM STDIN for > 10k rows)
rows = df.to_sql(CONN, "orders_processed", mode="append")
print(f"Written {rows:,} rows")

# Write — upsert
rows = df.to_sql(CONN, "orders_processed", mode="upsert", conflict_cols=["id"])
```

Node.js

```
const { read_sql, executeSql } = require('pardox/src/io');

const CONN = 'postgresql://pardox:secret@localhost:5432/mydb';

const df = read_sql(CONN, 'SELECT * FROM orders');
executeSql(CONN, 'CREATE TABLE IF NOT EXISTS orders_bak (id BIGINT, amount FLOAT)');

const rows = df.toSql(CONN, 'orders_bak', 'append');
console.log(`Written ${rows} rows`);

// Upsert
const rows2 = df.toSql(CONN, 'orders_bak', 'upsert', ['id']);
```

PHP

```
use PardoX\DataFrame;
use PardoX\IO;

$CONN = 'postgresql://pardox:secret@localhost:5432/mydb';

$df = IO::read_sql($CONN, 'SELECT * FROM orders');
IO::executeSql($CONN, 'CREATE TABLE IF NOT EXISTS orders_bak (id BIGINT, amount FLOAT)');

$rows = $df->to_sql($CONN, 'orders_bak', 'append');
echo "Written $rows rows\n";

// Upsert
$rows = $df->to_sql($CONN, 'orders_bak', 'upsert', ['id']);
```

Write modes

Mode	SQL generated

append
INSERT INTO table (cols) VALUES … Auto-upgrades to COPY FROM STDIN for > 10,000 rows

upsert
INSERT INTO table (cols) VALUES … ON CONFLICT (key_cols) DO UPDATE SET …

MySQL
Python

```
from pardox.io import read_mysql, execute_mysql

CONN = "mysql://pardox:secret@localhost:3306/mydb"

# Read
df = read_mysql(CONN, "SELECT * FROM products WHERE stock > 0")

# DDL
execute_mysql(CONN, "DROP TABLE IF EXISTS products_bak")
execute_mysql(CONN, """
    CREATE TABLE products_bak (
        id       BIGINT PRIMARY KEY,
        name     TEXT,
        price    DOUBLE,
        quantity DOUBLE
    )
""")

# Write
rows = df.to_mysql(CONN, "products_bak", mode="append")
rows = df.to_mysql(CONN, "products_bak", mode="replace")
rows = df.to_mysql(CONN, "products_bak", mode="upsert", conflict_cols=["id"])
```

Node.js

```
const { read_mysql, execute_mysql } = require('pardox/src/io');
```

```
const df = read_mysql(CONN, 'SELECT * FROM products');
execute_mysql(CONN, 'CREATE TABLE IF NOT EXISTS products_bak (id BIGINT, price DOUBLE)');

const rows = df.toMysql(CONN, 'products_bak', 'append');
const rows2 = df.toMysql(CONN, 'products_bak', 'upsert', ['id']);
```

PHP

```
$df = IO::read_mysql($CONN, 'SELECT * FROM products');
IO::executeMysql($CONN, 'CREATE TABLE IF NOT EXISTS products_bak (id BIGINT, price DOUBLE)');

$rows = $df->to_mysql($CONN, 'products_bak', 'append');
$rows = $df->to_mysql($CONN, 'products_bak', 'upsert', ['id']);
```

Write modes

Mode	SQL generated
append	INSERT INTO table (cols) VALUES (chunk of 1,000 rows) Auto-upgrades to LOAD DATA LOCAL INFILE for > 10,000 rows if local_infile=ON
replace	REPLACE INTO table (cols) VALUES (chunk of 1,000 rows)
upsert	INSERT INTO table (cols) VALUES … ON DUPLICATE KEY UPDATE col=VALUES(col)…

Info: ```sql
SET GLOBAL local_infile = 1;
-- or in my.cnf: local_infile=1

If disabled, PardoX automatically falls back to chunked INSERT and logs a notice to stderr.

SQL Server

Python

```python
from pardox.io import read_sqlserver, execute_sqlserver

CONN = "Server=localhost,1433;Database=mydb;UID=sa;PWD=MyPassword;TrustServerCertificate=Yes"

# Read
df = read_sqlserver(CONN, "SELECT TOP 5000 * FROM dbo.transactions")

# DDL
execute_sqlserver(CONN, "DROP TABLE IF EXISTS dbo.transactions_bak")
execute_sqlserver(CONN, """
    CREATE TABLE dbo.transactions_bak (
        id     BIGINT,
        amount FLOAT,
        region NVARCHAR(MAX)
    )
""")

# Write — batch INSERT 500 rows per statement
rows = df.to_sqlserver(CONN, "dbo.transactions_bak", mode="append")

# Upsert — MERGE INTO … WHEN MATCHED … WHEN NOT MATCHED
rows = df.to_sqlserver(CONN, "dbo.transactions_bak", mode="upsert", conflict_cols=["id"])
```

Node.js

```
const { read_sqlserver, execute_sqlserver } = require('pardox/src/io');

const df = read_sqlserver(CONN, 'SELECT TOP 1000 * FROM dbo.orders');
execute_sqlserver(CONN, 'DROP TABLE IF EXISTS dbo.orders_bak');

const rows = df.toSqlserver(CONN, 'dbo.orders_bak', 'append');
const rows2 = df.toSqlserver(CONN, 'dbo.orders_bak', 'upsert', ['id']);
```

PHP

```
$df = IO::read_sqlserver($CONN, 'SELECT TOP 1000 * FROM dbo.orders');
IO::executeSqlserver($CONN, 'DROP TABLE IF EXISTS dbo.orders_bak');

$rows = $df->to_sqlserver($CONN, 'dbo.orders_bak', 'append');
$rows = $df->to_sqlserver($CONN, 'dbo.orders_bak', 'upsert', ['id']);
```

Write modes

Mode	SQL generated
append	INSERT INTO table (cols) VALUES (r1),(r2),…,(r500) — 500 rows per statement
replace	Same multi-row INSERT
upsert	MERGE INTO table AS T USING (VALUES …) AS S (cols) ON (key) WHEN MATCHED … WHEN NOT MATCHED …

Warning: A known issue in tiberius v0.12 causes authentication failure when the SQL Server password contains !. Use passwords with only [A-Za-z0-9_\-@#$]. A fix is tracked for v0.3.2.

MongoDB

Python

```
from pardox.io import read_mongodb, execute_mongodb

CONN = "mongodb://admin:secret@localhost:27017"

# Read — "database.collection"
df = read_mongodb(CONN, "mydb.orders")
print(df.shape)

# Command (admin operations)
execute_mongodb(CONN, "mydb", '{"drop": "orders_archive"}')

# Write — append (insert_many 10k/batch, ordered:false)
rows = df.to_mongodb(CONN, "mydb.orders_archive", mode="append")

# Write — replace (drop collection, then re-insert)
rows = df.to_mongodb(CONN, "mydb.orders_archive", mode="replace")
```

Node.js

```
const { read_mongodb, execute_mongodb } = require('pardox/src/io');

const df = read_mongodb(CONN, 'mydb.orders');
execute_mongodb(CONN, 'mydb', '{"drop": "orders_archive"}');

const rows = df.toMongodb(CONN, 'mydb.orders_archive', 'append');
```

PHP

```
$df = IO::read_mongodb($CONN, 'mydb.orders');
IO::executeMongodb($CONN, 'mydb', '{"drop": "orders_archive"}');

$rows = $df->to_mongodb($CONN, 'mydb.orders_archive', 'append');
```

Write modes

Mode
Behavior

append
insert_many in batches of 10,000 documents, ordered: false

replace
Drops the collection, then insert_many in batches of 10,000

Bulk Write Performance

Database
Rows < 10k
Rows ≥ 10k (append)
Protocol

PostgreSQL
Multi-row INSERT
COPY FROM STDIN (auto)
Wire-level streaming

MySQL
1,000 rows/INSERT

LOAD DATA LOCAL INFILE (auto, if local_infile=ON)
In-memory CSV buffer

SQL Server
500 rows/INSERT
500 rows/INSERT
Multi-value INSERT

MongoDB
10,000 docs/batch
10,000 docs/batch
insert_many

PRDX Streaming to PostgreSQL (v0.3.2)
New in v0.3.2: Stream any .prdx file directly to PostgreSQL — without loading the file into RAM. Uses COPY FROM STDIN with block-by-block decompression.
Python

```
from pardox import write_sql_prdx

CONN = "postgresql://user:pass@localhost:5434/mydb"

rows = write_sql_prdx(
    "/data/ventas_150m.prdx",
    CONN,
    "ventas",          # table must exist with matching schema
    mode="append",
    conflict_cols=[],
    batch_rows=1_000_000
)
print(f"Streamed {rows:,} rows")
```

JavaScript

```
const { write_sql_prdx } = require('./pardox/src/io');

const rows = write_sql_prdx(
```

```
    '/data/ventas_150m.prdx',
    'postgresql://user:pass@localhost:5434/mydb',
    'ventas',
    'append',
    [],
    1000000
);
console.log(`Streamed ${rows.toLocaleString()} rows`);
```

PHP

```
use PardoX\IO;

$rows = IO::write_sql_prdx(
    '/data/ventas_150m.prdx',
    'postgresql://user:pass@localhost:5434/mydb',
    'ventas',
    'append',
    [],
    1000000
);
echo "Streamed $rows rows\n";
```

Memory characteristics

Approach	RAM used
df = read_prdx(path) then df.to_sql(...)	Entire file loaded (3.8 GB for 150M rows)
write_sql_prdx(path, ...)	O(one block × columns) — typically < 200 MB

Validated Results (v0.3.2)

PRDX → PostgreSQL — 150M rows (3.8 GB)

SDK

Rows

Time

Throughput

Protocol

Python

150,000,000

~490s

~306,000 rows/s

COPY FROM STDIN

JavaScript

150,000,000

~514s

~291,000 rows/s

COPY FROM STDIN

PHP

150,000,000

~1,032s

~145,000 rows/s

batch INSERT

CSV / DataFrame → PostgreSQL

Database
SDK
Result

PostgreSQL
Python
50,000 rows — COPY FROM STDIN

PostgreSQL
Node.js
50,000 rows — COPY FROM STDIN

MySQL
Python
50,000 rows — chunked batch (LOAD DATA disabled server-side)

MySQL
PHP
50,000 rows — chunked batch

SQL Server
all
⚠ Authentication fails with ! in password

MongoDB
—
Implemented; bulk benchmarks planned

User Guide — GPU Acceleration

GPU Acceleration

PardoX v0.3.2 includes a GPU Bitonic Sort pipeline powered by WebGPU compute shaders via the wgpu crate. The GPU engine is loaded dynamically from a separate library (libpardox_gpu.so), keeping it completely optional.

How It Works

```
df.sort_values("col", gpu=True)
        │
        ▼
  pardox_gpu_sort (Rust FFI)
        │
        ├── Load libpardox_gpu.so dynamically (libloading 0.8)
        ├── Extract Float64 column into contiguous buffer
        ├── Transfer buffer to GPU (VRAM)
        ├── Execute Bitonic Sort compute shader (WebGPU / wgpu 0.19)
        ├── Read sorted buffer back from GPU
        └── Apply row permutation to full DataFrame
        │
        ▼  (if GPU unavailable)
  CPU fallback — parallel Rust merge sort (Rayon)
```

The GPU sort works exclusively on Float64 columns. The result is applied back to the full DataFrame row order via an argsort + permutation step entirely in Rust.

Usage — Python

```python
# CPU sort (default)
sorted_df = df.sort_values("revenue", ascending=True)
sorted_df = df.sort_values("revenue", ascending=False)

# GPU Bitonic sort
sorted_df = df.sort_values("revenue", ascending=True, gpu=True)

print(f"Rows: {sorted_df.shape[0]:,}")
```

Usage — Node.js

```node.js
// CPU sort
```

```
const sortedDf = df.sortValues('revenue', true);

// GPU sort
const sortedDf = df.sortValues('revenue', true, true);  // (by, ascending, gpu)
```

Usage — PHP

```
// CPU sort
$sorted = $df->sort_values('revenue', true);

// GPU sort
$sorted = $df->sort_values('revenue', true, true);  // ($by, $ascending, $gpu)
```

CPU Fallback

If a compatible GPU is not available or the wgpu backend fails to initialize:

PardoX logs [PardoX GPU Sort] GPU not available, using CPU sort. to stderr.
The CPU parallel merge sort (Rayon) is used transparently.
The result is identical — only the execution path differs.

This means you can safely call gpu=True in all environments without adding error handling.

GPU Requirements

Requirement	Details
API	WebGPU via wgpu 0.19
Backends	Vulkan (Linux / Windows), Metal (macOS), DX12 (Windows)

VRAM
Minimum ~256 MB for 50k rows (Float64 = 400 KB; permutation overhead ~4 MB)

Adapter
Any discrete or integrated GPU with Vulkan / Metal / DX12 support

Note: PardoX uses the native Rust wgpu crate, which targets Vulkan/Metal/DX12 directly. It does not require a browser or WebAssembly environment.

Dynamic Library Loading
The GPU engine is shipped as a separate shared library (libpardox_gpu.so / .dylib / .dll) loaded at runtime via libloading 0.8. The main libpardox.so will function normally even if libpardox_gpu.so is absent — GPU calls simply fall back to CPU.

Performance
GPU sort shines for very large Float64 columns. For small datasets (< 100k rows), the overhead of GPU transfer and shader dispatch may outweigh the benefit.

Rows
CPU sort
GPU Bitonic sort

50k
~5ms
~15ms (transfer overhead dominates)

1M
~120ms

~40ms

10M
~1.4s
~280ms

Tip: Use gpu=True when sorting columns with more than 500,000 rows. For smaller datasets, the default CPU path is faster due to lower overhead.

User Guide — ML Integration

ML Integration — Zero-Copy NumPy Bridge

PardoX v0.3.2 implements the NumPy __array__ protocol, allowing direct zero-copy access from PardoX Series to NumPy arrays. No data is copied — the NumPy array points directly into the Rust HyperBlock buffer.

Zero-Copy NumPy Conversion

```
import numpy as np
import pardox as px

df = px.read_csv("features.csv")

# Direct pointer into Rust buffer — no data copy
arr = np.array(df["price"])
print(arr.dtype)    # float64
print(arr.shape)    # (50000,)
print(arr.mean())   # identical to df["price"].mean()
```

Success: np.array(df['col']) invokes the __array__ dunder, which calls pardox_get_f64_buffer — a Rust function that returns a direct pointer into the HyperBlock's Float64 buffer. NumPy wraps this pointer in an array without allocating new memory.

Warning: The NumPy array is valid as long as the parent DataFrame is alive. Do not hold onto arr after df goes out of scope or is freed.

Scikit-Learn Integration

```
import numpy as np
import pardox as px
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

df = px.read_csv("training_data.csv")
df.cast("quantity", "Float64")
df.fillna(0.0)

# Build feature matrix and target via zero-copy
X = np.column_stack([
    np.array(df["price"]),
```

```
    np.array(df["quantity"]),
    np.array(df["discount"]),
])
y = np.array(df["tax"])

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train
model = LinearRegression()
model.fit(X_scaled, y)

print(f"Coefficients: {model.coef_}")
print(f"Intercept:    {model.intercept_:.4f}")
```

PyTorch Integration

```
import torch
import numpy as np
import pardox as px

df = px.read_csv("embeddings.csv")

# Zero-copy from PardoX → NumPy → PyTorch tensor
arr    = np.array(df["embedding_dim_0"])
tensor = torch.from_numpy(arr)

print(tensor.dtype)    # torch.float64
print(tensor.shape)    # torch.Size([50000])
```

Min-Max Scaling (Built-In)

For normalizing features before ML, PardoX provides a native Min-Max Scaler that runs entirely in Rust:

```
# Normalize to [0, 1] — returns new DataFrame with 'result_minmax'
normed_df = df.min_max_scale("price")
normed_arr = np.array(normed_df["result_minmax"])
```

```
print(normed_arr.min())   # ~0.0
print(normed_arr.max())   # ~1.0
```

This is equivalent to sklearn.preprocessing.MinMaxScaler but operates at native Rust speed with no intermediate Python allocations.

Supported Column Types

The zero-copy __array__ bridge works on Float64 columns. For Int64 columns, cast first:

```
df.cast("quantity", "Float64")
arr = np.array(df["quantity"])   # now Float64
```

Utf8 (string) columns cannot be zero-copied to NumPy. Use df.to_dict() or df.unique() to access string data.

No NumPy Import Inside PardoX

PardoX itself does not import NumPy. The __array__ protocol is implemented entirely through ctypes pointer arithmetic. NumPy is only involved at the user call site when np.array(series) is invoked.

This means PardoX works in environments where NumPy is not installed, and NumPy integration is opt-in.

SDK Documentation — Python

PardoX Python SDK Documentation

Installation

```
pip install pardox
```

Or install from source:

```
cd pardox_project
pip install -e .
```

Quick Start

```
import pardox as px

# Create DataFrame from dictionary
df = px.DataFrame([
    {"name": "Alice", "age": 30, "score": 85.5},
    {"name": "Bob", "age": 25, "score": 92.0},
    {"name": "Charlie", "age": 35, "score": 78.5}
])

print(df.head())
```

API Reference

DataFrame Creation

px.DataFrame(data)

Create a DataFrame from a list of dictionaries.

```
import pardox as px

# From list of dicts
df = px.DataFrame([
    {"name": "Alice", "age": 30},
    {"name": "Bob", "age": 25}
])

# From dict with lists
df = px.DataFrame({
    "name": ["Alice", "Bob", "Charlie"],
    "age": [30, 25, 35]
```

```
})
```

I/O Operations

px.read_csv(path, schema=None, **kwargs)

Read a CSV file into a DataFrame.

```
import pardox as px

# Basic CSV reading
df = px.read_csv("data.csv")

# With schema specification
df = px.read_csv("data.csv", schema={
    "id": "Int64",
    "name": "Utf8",
    "amount": "Float64"
})

# With custom delimiter
df = px.read_csv("data.csv", delimiter=";")
```

px.read_sql(conn_string, query)

Execute a SQL query and return results as DataFrame.

```
import pardox as px

# PostgreSQL
df = px.read_sql(
    "postgresql://user:password@localhost:5432/mydb",
    "SELECT * FROM sales WHERE amount > 1000"
)
```

px.read_prdx(path)

Read a PardoX binary file (.prdx).

```
import pardox as px

df = px.read_prdx("data.prdx")
```

DataFrame Methods

df.head(n=5)
Return the first n rows.
# First 10 rows
print(df.head(10))

df.tail(n=5)
Return the last n rows.
# Last 5 rows
print(df.tail())

df.shape
Return the dimensions of the DataFrame (rows, columns).
rows, cols = df.shape
print(f"DataFrame has {rows} rows and {cols} columns")

df.dtypes
Return the data types of each column.
print(df.dtypes)
# Output: {'name': 'Utf8', 'age': 'Int64', 'score': 'Float64'}

df.iloc[start:end]
Slice rows by position.
# Rows 10-20
subset = df.iloc[10:20]

df.show(n=10)
Display an ASCII table representation.
df.show(10)

Column Operations
Column Selection
# Select single column (returns Series)
ages = df["age"]

Select multiple columns

```
subset = df[["name", "score"]]
```

Column Assignment

```
# Add new column
df["new_col"] = df["age"] * 2

# Or use assign
df = df.assign(new_col=lambda x: x["age"] * 2)
```

Arithmetic Operations

df.add(col_a, col_b)

Add two columns.

```
# Create new column with sum
df = df.add("amount", "tax")
# Result stored in "result_math_add" column
```

df.sub(col_a, col_b)

Subtract columns.

```
df = df.sub("price", "discount")
```

df.std(col)

Calculate standard deviation.

```
std_value = df.std("score")
```

df.min_max_scale(col)

Normalize column to [0, 1] range.

```
df = df.min_max_scale("amount")
```

Filtering

Boolean Filtering

```
# Filter by condition
filtered = df[df["age"] > 25]

# Multiple conditions
filtered = df[(df["age"] > 25) & (df["score"] > 80)]
```

Aggregations

Available on Series objects:

```
# Column as Series
col = df["score"]

# Aggregations
total = col.sum()
average = col.mean()
maximum = col.max()
minimum = col.min()
count = col.count()
std_dev = col.std()
```

Sorting

df.sort_values(by, ascending=True, gpu=False)

Sort DataFrame by column values.

```
# Sort by name ascending
df_sorted = df.sort_values("name")

# Sort by amount descending
df_sorted = df.sort_values("amount", ascending=False)

# Use GPU for large datasets
df_sorted = df.sort_values("amount", gpu=True)
```

Joins

df.join(other, on, how='inner')

Join with another DataFrame.

```
# Inner join on client_id
result = df.join(
    other=clients_df,
    on="client_id"
)

# Left join
result = df.join(
    other=clients_df,
```

```
    on="client_id",
    how="left"
)
```

Data Cleaning

df.fillna(value)

Fill null values.

```
# Fill numeric columns with 0
df = df.fillna(0.0)

# Fill with specific value
df = df.fillna({"score": 50.0, "name": "Unknown"})
```

df.round(decimals)

Round numeric columns.

```
# Round to 2 decimal places
df = df.round(2)
```

Type Casting

df.cast(column, target_type)

Convert column to different type.

```
# Convert string to integer
df = df.cast("age", "Int64")

# Convert to float
df = df.cast("price", "Float64")
```

Export

df.to_csv(path, **kwargs)

Export to CSV.

```
df.to_csv("output.csv")

# With options
df.to_csv("output.csv", delimiter=";")
```

df.to_prdx(path)

Export to PardoX binary format.

```
df.to_prdx("data.prdx")
```

```
df.to_sql(conn_string, table, mode='append', conflict_cols=None)
```

Write to SQL table.

```
# Append to table
df.to_sql(
    "postgresql://user:pass@localhost/mydb",
    "sales",
    mode="append"
)

# Upsert with conflict handling
df.to_sql(
    "postgresql://user:pass@localhost/mydb",
    "sales",
    mode="upsert",
    conflict_cols=["id"]
)
```

Series API

The Series class represents a single column:

```
s = df["name"]

# Arithmetic
s2 = s + " (copy)"

# Comparisons (return boolean Series)
mask = s == "Alice"

# String operations
upper = s.upper()
lower = s.lower()
```

Database Support

PostgreSQL

```
import pardox as px
```

```
# Read from PostgreSQL
df = px.read_sql(
    "postgresql://user:password@host:5432/db",
    "SELECT * FROM table"
)

# Write to PostgreSQL
df.to_sql(
    "postgresql://user:password@host:5432/db",
    "table_name",
    mode="append"  # or "upsert"
)
```

MySQL

```
# Read from MySQL
df = px.read_mysql(
    "mysql://user:password@host:3306/db",
    "SELECT * FROM table"
)

# Write to MySQL
df.to_mysql(
    "mysql://user:password@host:3306/db",
    "table_name",
    mode="append"
)
```

SQL Server

```
# Read from SQL Server
df = px.read_sqlserver(
    "Server=host;Database=db;User Id=user;Password=pass;",
    "SELECT * FROM table"
)

# Write to SQL Server
df.to_sqlserver(
    "Server=host;Database=db;User Id=user;Password=pass;",
    "table_name"
)
```

MongoDB

```
# Read from MongoDB
df = px.read_mongodb(
    "mongodb://user:password@host:27017",
    "database.collection"
)

# Write to MongoDB
df.to_mongodb(
    "mongodb://user:password@host:27017",
    "database.collection",
    mode="append"  # or "replace"
)
```

Performance Tips

Use .prdx format for repeated reads - it's much faster than CSV
Use GPU sorting for large datasets: df.sort_values(col, gpu=True)
Batch SQL writes: Use mode="append" for bulk inserts
Use fillna() before computations to handle missing values

Error Handling

```
import pardox as px

try:
    df = px.read_csv("nonexistent.csv")
except FileNotFoundError as e:
    print(f"File not found: {e}")
except Exception as e:
    print(f"Error: {e}")
```

Binary Format (.prdx)

PardoX's native binary format provides:
- ~4.6 GB/s read throughput
- Columnar storage (HyperBlock)
- Automatic compression

```
# Write
```

```
df.to_prdx("data.prdx")

# Read (much faster than CSV)
df = px.read_prdx("data.prdx")
```

SDK Documentation — Node.js

PardoX JavaScript/Node.js SDK Documentation

Installation

```
npm install @pardox/pardox
```

Or install from source:

```
git clone https://github.com/your-repo/pardox-js.git
cd pardox-js
npm install
```

Requirements

Node.js 18 or higher

koffi (npm install koffi)

Quick Start

```
const { DataFrame } = require('@pardox/pardox');

// Create DataFrame from array
const df = DataFrame.fromArray([
    { name: "Alice", age: 30, score: 85.5 },
    { name: "Bob", age: 25, score: 92.0 },
    { name: "Charlie", age: 35, score: 78.5 }
]);

df.show();
```

API Reference

DataFrame Creation

DataFrame.fromArray(data)

Create a DataFrame from an array of objects.

```
const { DataFrame } = require('@pardox/pardox');

// From array of objects
const df = DataFrame.fromArray([
    { name: "Alice", age: 30 },
    { name: "Bob", age: 25 }
```

```
]);

// Empty DataFrame
const df = new DataFrame();
```

I/O Operations

DataFrame.read_csv(path, schema?)

Read a CSV file into a DataFrame.

```
const { DataFrame } = require('@pardox/pardox');

// Basic CSV reading
const df = DataFrame.read_csv('./data.csv');

// With schema specification
const df = DataFrame.read_csv('./data.csv', {
    id: 'Int64',
    name: 'Utf8',
    amount: 'Float64'
});
```

DataFrame.read_sql(connectionString, query)

Execute a SQL query and return results as DataFrame.

```
const { DataFrame } = require('@pardox/pardox');

// PostgreSQL
const df = DataFrame.read_sql(
    'postgresql://user:password@localhost:5432/mydb',
    'SELECT * FROM sales WHERE amount > 1000'
);
```

DataFrame.read_mysql(connectionString, query)

Read from MySQL.

```
const { DataFrame } = require('@pardox/pardox');

const df = DataFrame.read_mysql(
    'mysql://user:password@localhost:3306/mydb',
    'SELECT * FROM users'
);
```

DataFrame.read_sqlserver(connectionString, query)
Read from SQL Server.

```
const { DataFrame } = require('@pardox/pardox');

const df = DataFrame.read_sqlserver(
    'Server=localhost;Database=mydb;User Id=user;Password=pass;',
    'SELECT * FROM orders'
);
```

DataFrame.read_mongodb(connectionString, dbDotCollection)
Read from MongoDB.

```
const { DataFrame } = require('@pardox/pardox');

const df = DataFrame.read_mongodb(
    'mongodb://user:password@localhost:27017',
    'mydb.collection'
);
```

DataFrame Methods

df.head(n=5)
Return the first n rows.

```
// First 10 rows
const head = df.head(10);
```

df.tail(n=5)
Return the last n rows.

```
const tail = df.tail(5);
```

df.shape
Return the dimensions of the DataFrame.

```
const [rows, cols] = df.shape;
console.log(`Rows: ${rows}, Cols: ${cols}`);
```

df.columns

Return the list of column names.

```
const cols = df.columns;
console.log(cols); // ['name', 'age', 'score']
```

df.dtypes

Return the data types of each column.

```
const types = df.dtypes;
// { name: 'Utf8', age: 'Int64', score: 'Float64' }
```

df.iloc(start, end)

Slice rows by position.

```
// Rows 10-20
const subset = df.iloc(10, 20);
```

df.show(n=10)

Display an ASCII table representation.

```
df.show(10);
```

Column Operations

Column Selection (Proxy-based)

```
// Select single column (returns Series)
const ages = df.age;
const names = df['name'];

// Using method
const col = df.column('age');
```

Column Assignment

```
// Add new column
df.newCol = df.age.map(x => x * 2);

// Or using setColumn
df._setColumn('double_age', df.age.mul(2));
```

Arithmetic Operations

df.add(colA, colB)
Add two columns.

```
// Create new column with sum
const result = df.add('amount', 'tax');
// Result stored in "result_math_add" column
```

df.sub(colA, colB)
Subtract columns.

```
const result = df.sub('price', 'discount');
```

df.std(col)
Calculate standard deviation.

```
const stdValue = df.std('score');
```

df.minMaxScale(col)
Normalize column to [0, 1] range.

```
const scaled = df.minMaxScale('amount');
```

Filtering
Boolean Filtering

```
// Filter using a mask Series
const mask = df.age.gt(25);
const filtered = df.filter(mask);

// Direct comparison returns a mask
const filtered2 = df.filter(df.amount.gt(1000));
```

Aggregations
Available on Series objects:

```
const col = df.score;

// Aggregations
const total = col.sum();
const average = col.mean();
const maximum = col.max();
const minimum = col.min();
```

```
const count = col.count();
const stdDev = col.std();
```

Sorting

df.sortValues(by, ascending=true, gpu=false)

Sort DataFrame by column values.

```
// Sort by name ascending
const sorted = df.sortValues('name');

// Sort by amount descending
const sorted = df.sortValues('amount', false);

// Use GPU for large datasets
const sorted = df.sortValues('amount', false, true);
```

Joins

df.join(other, { on, leftOn, rightOn })

Join with another DataFrame.

```
// Inner join on client_id
const result = df.join(clients_df, { on: 'client_id' });

// With different column names
const result = df.join(clients_df, {
    leftOn: 'customer_id',
    rightOn: 'id'
});
```

Data Cleaning

df.fillna(val)

Fill null values.

```
df.fillna(0.0);
```

df.round(decimals)

Round numeric columns.

```
df.round(2);
```

Type Casting

df.cast(col, targetType)

Convert column to different type.

```
df.cast('age', 'Int64');
df.cast('price', 'Float64');
```

Export

df.toCsv(path)

Export to CSV.

```
df.toCsv('./output.csv');
```

df.toPrdx(path)

Export to PardoX binary format.

```
df.toPrdx('./data.prdx');
```

df.toSql(connectionString, table, mode, conflictCols)

Write to SQL table.

```
// Append to table
df.toSql('postgresql://user:pass@localhost/mydb', 'sales', 'append');

// Upsert with conflict handling
df.toSql('postgresql://user:pass@localhost/mydb', 'sales', 'upsert', ['id']);
```

df.toMysql(connectionString, table, mode, conflictCols)

Write to MySQL table.

```
df.toMysql('mysql://user:pass@localhost/mydb', 'users', 'append');
```

df.toSqlserver(connectionString, table, mode, conflictCols)

Write to SQL Server.

```
df.toSqlserver('Server=localhost;Database=mydb;User Id=user;Password=pass;', 'orders');
```

df.toMongodb(connectionString, dbDotCollection, mode)

Write to MongoDB.

```
df.toMongodb('mongodb://user:pass@localhost:27017', 'mydb.collection', 'append');
```

Series API

The Series class represents a single column:

```
const s = df.name;

// Arithmetic
const s2 = s.add(s);        // vectorized add
const s2 = s.sub(s);        // vectorized sub
const s2 = s.mul(2);        // multiply by scalar
const s2 = s.div(2);        // divide by scalar
const s2 = s.mod(2);        // modulo

// Comparisons (return boolean Series)
const mask = s.eq('Alice');
const mask = s.neq('Bob');
const mask = s.gt(25);
const mask = s.gte(18);
const mask = s.lt(65);
const mask = s.lte(100);

// Aggregations
const sum = s.sum();
const mean = s.mean();
const min = s.min();
const max = s.max();
const count = s.count();
const std = s.std();
```

Observer / Export Methods

df.toJson(limit?)

Serialize to JSON string.

```
const json = df.toJson(100);
```

df.toDict()

Convert to array of objects.

```
const arr = df.toDict();
// [{ name: 'Alice', age: 30 }, ...]
```

df.toList()

Convert to array of arrays.

```
const arr = df.toList();
// [['Alice', 30], ['Bob', 25], ...]
```

df.valueCounts(col)

Get frequency of unique values.

```
const counts = df.valueCounts('category');
// { Electronics: 150, Clothing: 100, ... }
```

df.unique(col)

Get unique values.

```
const unique = df.unique('name');
// ['Alice', 'Bob', 'Charlie', ...]
```

Functional API

The SDK also provides functional-style I/O helpers:

```
const { read_csv, read_sql, read_mysql, read_sqlserver, read_mongodb } = require('@pardox/pardox');

// Read CSV
const df = read_csv('./data.csv');

// Read from SQL
const df = read_sql('postgresql://...', 'SELECT * FROM users');

// Execute SQL (returns row count)
const { executeSql } = require('@pardox/pardox');
const count = executeSql('postgresql://...', 'INSERT INTO ...');
```

Error Handling

```
const { DataFrame } = require('@pardox/pardox');

try {
    const df = DataFrame.read_csv('./nonexistent.csv');
} catch (e) {
    console.error('Error:', e.message);
```

```
}

// Check if DataFrame is empty
if (df.shape[0] === 0) {
    console.log('DataFrame is empty');
}
```

Memory Management

The DataFrame manages Rust memory automatically. However, you can explicitly free memory:

```
const df = DataFrame.read_csv('./large_file.csv');

// Do operations...

// Free memory when done (optional - GC will also free)
df._free();
```

Binary Format (.prdx)

PardoX's native binary format provides:

- ~4.6 GB/s read throughput
- Columnar storage (HyperBlock)
- Automatic compression

```
const { DataFrame } = require('@pardox/pardox');

// Write
df.toPrdx('./data.prdx');

// Read (much faster than CSV)
const df = DataFrame.read_prdx('./data.prdx');
```

Performance Tips

Use .prdx format for repeated reads - it's much faster than CSV
Use GPU sorting for large datasets: df.sortValues(col, false, true)
Batch SQL writes with toSql(conn, table, 'append')
Use fillna() before computations to handle missing values
Reuse DataFrames when possible instead of creating new ones

TypeScript Support

The SDK includes TypeScript definitions. Import with types:

```
import { DataFrame, Series } from '@pardox/pardox';

const df: DataFrame = DataFrame.read_csv('./data.csv');
const col: Series = df.name;
const count: number = col.count();
```

Platform Support

The SDK automatically detects your platform and loads the correct binary:

Linux x64 (libpardox.so)
macOS Intel (libpardox.dylib)
macOS Apple Silicon (libpardox.dylib)
Windows x64 (pardox.dll)

Binaries are located in ./libs/ directory.

SDK Documentation — PHP

PardoX PHP SDK Documentation
Installation
composer require pardox/pardox-php

Or install manually:
git clone https://github.com/your-repo/pardox-php.git
cd pardox-php
composer install

Requirements

PHP 8.1 or higher
FFI extension enabled (ffi.enable=1 in php.ini)

Quick Start

```
 "Alice", "age" => 30, "score" => 85.5],
    ["name" => "Bob", "age" => 25, "score" => 92.0],
    ["name" => "Charlie", "age" => 35, "score" => 78.5]
];

$df = new DataFrame($data);

echo $df->show();
```

API Reference
DataFrame Creation
new DataFrame(array $data)
Create a DataFrame from an array of associative arrays.

```
 "Alice", "age" => 30],
    ["name" => "Bob", "age" => 25]
]);

// Empty DataFrame
$df = new DataFrame();
```

I/O Operations

DataFrame::read_csv(string $path, ?array $schema = null): self

Read a CSV file into a DataFrame.

```
 "Int64",
    "name" => "Utf8",
    "amount" => "Float64"
]);
```

DataFrame::read_sql(string $connString, string $query): self

Execute a SQL query and return results as DataFrame.

```
 1000"
);
```

IO::read_prdx(string $path): array

Read a PardoX binary file (.prdx).

head(int $n = 5): DataFrame`

Return the first n rows.

```php
head(10);
echo $head->show();
```

$df->tail(int $n = 5): DataFrame

Return the last n rows.

```
tail(5);
```

$df->shape: array

Return the dimensions of the DataFrame.

```
shape;  // [rows, cols]
echo "Rows: {$shape[0]}, Cols: {$shape[1]}";
```

$df->columns: array

Return the list of column names.

```
columns;
print_r($columns);
```

$df->dtypes: array
Return the data types of each column.
dtypes;
// ['name' => 'Utf8', 'age' => 'Int64', 'score' => 'Float64']
print_r($types);

$df->iloc(int $start, int $end): DataFrame
Slice rows by position.
iloc(10, 20);

$df->show(int $n = 10): string
Display an ASCII table representation.
show(10);

Column Operations
Column Selection
offsetGet("name");

Column Assignment
add(string $colA, string $colB): DataFrame`

Add two columns.

```php
add("amount", "tax");
// Result stored in "result_math_add" column

$df->sub(string $colA, string $colB): DataFrame
Subtract columns.
sub("price", "discount");

$df->std(string $col): float
Calculate standard deviation.
```

std("score");

$df->minMaxScale(string $col): DataFrame
Normalize column to [0, 1] range.
minMaxScale("amount");

Filtering
$df->filter(Series $mask): DataFrame
Filter using a boolean Series.
gt(25);
$filtered = $df->filter($mask);

Aggregations
Available on Series objects:
sum();
$average = $col->mean();
$maximum = $col->max();
$minimum = $col->min();
$count = $col->count();
$stdDev = $col->std();

Sorting
$df->sortValues(string $by, bool $ascending = true, bool $gpu = false): DataFrame
Sort DataFrame by column values.
sortValues("name");

// Sort by amount descending
$sorted = $df->sortValues("amount", false);

// Use GPU for large datasets
$sorted = $df->sortValues("amount", false, true);

Joins
$df->join(DataFrame $other, array $options): DataFrame
Join with another DataFrame.
join($clients_df, [

```
    "on" => "client_id"
]);

// With different column names
$result = $df->join($clients_df, [
    "leftOn" => "customer_id",
    "rightOn" => "id"
]);
```

Data Cleaning

$df->fillna(float $value): self

Fill null values.

```
fillna(0.0);
```

$df->round(int $decimals): self

Round numeric columns.

```
round(2);
```

Type Casting

$df->cast(string $column, string $targetType): self

Convert column to different type.

```
cast("age", "Int64");

// Convert to float
$df->cast("price", "Float64");
```

Export

$df->to_csv(string $path): bool

Export to CSV.

```
to_csv("output.csv");
```

$df->to_prdx(string $path): bool

Export to PardoX binary format.

```
to_prdx("data.prdx");
```

$df->to_sql(string $connString, string $table, string $mode = 'append', array $conflictCols = []): int

Write to SQL table.

```
to_sql(
    "postgresql://user:pass@localhost/mydb",
    "sales",
    "append"
);

// Upsert with conflict handling
$rows = $df->to_sql(
    "postgresql://user:pass@localhost/mydb",
    "sales",
    "upsert",
    ["id"]
);
```

Series API

The Series class represents a single column:

```
add($s);  // vectorized add
$s2 = $s->sub($s);  // vectorized sub
$s2 = $s->mul(2);   // multiply by scalar
$s2 = $s->div(2);   // divide by scalar

// Comparisons (return boolean Series)
$mask = $s->eq("Alice");
$mask = $s->neq("Bob");
$mask = $s->gt(25);
$mask = $s->gte(18);
$mask = $s->lt(65);
$mask = $s->lte(100);

// Aggregations
$sum = $s->sum();
$mean = $s->mean();
$min = $s->min();
$max = $s->max();
$count = $s->count();
$std = $s->std();
```

Database Support

PostgreSQL

```
to_sql(
    "postgresql://user:password@host:5432/db",
    "table_name",
    "append"  // or "upsert"
);
```

MySQL

```
to_mysql(
    "mysql://user:password@host:3306/db",
    "table_name",
    "append"  // or "replace", "upsert"
);
```

SQL Server

```
to_sqlserver(
    "Server=host;Database=db;User Id=user;Password=pass;",
    "table_name",
    "append"
);
```

MongoDB

```
to_mongodb(
    "mongodb://user:password@host:27017",
    "database.collection",
    "append"  // or "replace"
);
```

Observers / Export Methods

$df->to_json(): string

Serialize to JSON string.

```
to_json();
```

$df->to_dict(): array

Convert to array of objects.

```
to_dict();
// [["name"=>"Alice", "age"=>30], ...]
```

$df->tolist(): array

Convert to array of arrays.

```
tolist();
// [["Alice", 30], ["Bob", 25], ...]
```

$df->value_counts(string $col): array

Get frequency of unique values.

```
value_counts("category");
// ["Electronics" => 150, "Clothing" => 100, ...]
```

$df->unique(string $col): array

Get unique values.

```
unique("name");
// ["Alice", "Bob", "Charlie", ...]
```

Error Handling

```
getMessage();
}

// Check if DataFrame is empty
if ($df->shape[0] === 0) {
    echo "DataFrame is empty";
}
```

Binary Format (.prdx)

PardoX's native binary format provides:

- ~4.6 GB/s read throughput
- Columnar storage (HyperBlock)
- Automatic compression

```
to_prdx("data.prdx");

// Read (much faster than CSV)
```

```
$data = IO::read_prdx("data.prdx");
$df = new DataFrame($data);
```

Performance Tips

Use .prdx format for repeated reads
Use GPU sorting for large datasets: sortValues("col", false, true)
Batch SQL writes with mode="append"
Use fillna() before computations
Enable FFI caching in production:

```
// In your bootstrap
if (function_exists('opcache_get_status')) {
    opcache_compile_file(__DIR__ . '/vendor/autoload.php');
}
```

SDK Documentation — Database Integration

PardoX — Database Integration Guide
Version: v0.3.1
SDKs covered: Python · Node.js · PHP

Overview
PardoX uses native Rust drivers for all database operations. There is no Python/Node.js/PHP database library involved — the host language passes a connection string and a pointer to the DataFrame; the Rust core handles the entire connection, serialization, and transfer. This means:

Zero Python/JS/PHP dependencies for database I/O
No ORM or query builder overhead
Bulk write paths operate at the wire level (COPY, LOAD DATA, batch INSERT)

Supported Databases

Database
Rust driver
Read
Write
Execute DDL/DML

PostgreSQL
tokio-postgres

MySQL
mysql v25

SQL Server
tiberius v0.12

MongoDB
mongodb v2.8

Connection String Formats
PostgreSQL
postgresql://user:password@host:port/database

Example:
postgresql://pardox:pardox123@localhost:5432/pardox_test

MySQL
mysql://user:password@host:port/database

Example:
mysql://pardox:pardoxpassword@localhost:3306/pardox_test

SQL Server (ADO.NET format)
Server=host,port;Database=db;UID=user;PWD=password;TrustServerCertificate=Yes

Example:

Server=localhost,1433;Database=mydb;UID=sa;PWD=MyPassword;TrustServerCertificate=Yes

Note: Avoid special characters (particularly !) in SQL Server passwords. See Known Limitations.

MongoDB
mongodb://user:password@host:port

Example:
mongodb://admin:secret@localhost:27017

PostgreSQL
Python

```
from pardox.io import read_sql, execute_sql
import pardox

# Read
df = read_sql("postgresql://pardox:pardox123@localhost:5432/mydb", "SELECT * FROM orders")

# Execute DDL / DML
execute_sql("postgresql://...", "DROP TABLE IF EXISTS orders")
execute_sql("postgresql://...", "CREATE TABLE orders (id BIGINT, amount FLOAT)")

# Write — append (auto-activates COPY FROM STDIN for > 10,000 rows)
rows = df.to_sql("postgresql://...", "orders", mode="append")

# Write — upsert (INSERT … ON CONFLICT DO UPDATE)
rows = df.to_sql("postgresql://...", "orders", mode="upsert", conflict_cols=["id"])
```

Node.js

```
const { read_sql, executeSql } = require('./pardox/src/io');
const { DataFrame } = require('./pardox/src/DataFrame');
```

```
// Read
const df = read_sql('postgresql://pardox:pardox123@localhost:5432/mydb', 'SELECT * FROM orders');

// Execute DDL / DML
executeSql('postgresql://...', 'DROP TABLE IF EXISTS orders');
executeSql('postgresql://...', 'CREATE TABLE orders (id BIGINT, amount FLOAT)');

// Write — append
const rows = df.toSql('postgresql://...', 'orders', 'append');

// Write — upsert
const rows = df.toSql('postgresql://...', 'orders', 'upsert', ['id']);
```

PHP

```
use PardoX\DataFrame;
use PardoX\IO;

// Read
$df = IO::read_sql('postgresql://pardox:pardox123@localhost:5432/mydb', 'SELECT * FROM orders');

// Execute DDL / DML
IO::executeSql('postgresql://...', 'DROP TABLE IF EXISTS orders');
IO::executeSql('postgresql://...', 'CREATE TABLE orders (id BIGINT, amount FLOAT)');

// Write — append
$rows = $df->to_sql('postgresql://...', 'orders', 'append');

// Write — upsert
$rows = $df->to_sql('postgresql://...', 'orders', 'upsert', ['id']);
```

Write Modes — PostgreSQL

Mode

Behavior

append
INSERT INTO … VALUES …. Auto-upgrades to COPY FROM STDIN when rows > 10,000.

upsert
INSERT … ON CONFLICT (cols) DO UPDATE SET …. Requires conflict_cols.

Bulk Performance — PostgreSQL
For DataFrames with more than 10,000 rows and mode="append", the Rust core automatically switches to COPY FROM STDIN. The CSV data is serialized in memory and streamed directly into PostgreSQL — no temp files.

MySQL
Python
```
from pardox.io import read_mysql, execute_mysql
import pardox

CONN = "mysql://pardox:pardoxpassword@localhost:3306/mydb"

# Read
df = read_mysql(CONN, "SELECT * FROM sales")

# Execute DDL / DML
execute_mysql(CONN, "DROP TABLE IF EXISTS sales")
execute_mysql(CONN, (
    "CREATE TABLE sales ("
    "id BIGINT, amount DOUBLE, category TEXT)"
))

# Write — append
rows = df.to_mysql(CONN, "sales", mode="append")

# Write — upsert (ON DUPLICATE KEY UPDATE)
rows = df.to_mysql(CONN, "sales", mode="upsert", conflict_cols=["id"])
```

Node.js

```
const { read_mysql, execute_mysql } = require('./pardox/src/io');

const CONN = 'mysql://pardox:pardoxpassword@localhost:3306/mydb';

// Read
const df = read_mysql(CONN, 'SELECT * FROM sales');

// Execute DDL / DML
execute_mysql(CONN, 'DROP TABLE IF EXISTS sales');
execute_mysql(CONN, 'CREATE TABLE sales (id BIGINT, amount DOUBLE, category TEXT)');

// Write — append
const rows = df.toMysql(CONN, 'sales', 'append');

// Write — replace (DELETE existing rows then INSERT)
const rows = df.toMysql(CONN, 'sales', 'replace');

// Write — upsert
const rows = df.toMysql(CONN, 'sales', 'upsert', ['id']);
```

PHP

```
use PardoX\DataFrame;
use PardoX\IO;

$CONN = 'mysql://pardox:pardoxpassword@localhost:3306/mydb';

// Read
$df = IO::read_mysql($CONN, 'SELECT * FROM sales');

// Execute DDL / DML
IO::executeMysql($CONN, 'DROP TABLE IF EXISTS sales');
IO::executeMysql($CONN, 'CREATE TABLE IF NOT EXISTS sales (id BIGINT, amount DOUBLE,
category TEXT)');

// Write — append
$rows = $df->to_mysql($CONN, 'sales', 'append');

// Write — upsert
$rows = $df->to_mysql($CONN, 'sales', 'upsert', ['id']);
```

Write Modes — MySQL

Mode	Behavior
append	INSERT INTO … VALUES … in chunks of 1,000 rows per statement. Auto-upgrades to LOAD DATA LOCAL INFILE when rows > 10,000 and the server has local_infile=ON.
replace	REPLACE INTO … VALUES … in chunks of 1,000 rows.
upsert	INSERT … ON DUPLICATE KEY UPDATE. Requires conflict_cols.

Bulk Performance — MySQL

When local_infile is enabled on the MySQL server, appending more than 10,000 rows automatically uses LOAD DATA LOCAL INFILE. The CSV payload is built in memory and delivered via the MySQL binary protocol — no disk files. If the server has local_infile=OFF (ERROR 3948), the Rust core falls back to the 1,000-row chunked INSERT automatically, logging a notice to stderr.

SQL Server

Python

```
from pardox.io import read_sqlserver, execute_sqlserver
import pardox

CONN = "Server=localhost,1433;Database=mydb;UID=sa;PWD=MyPassword;TrustServerCertificate=Yes"
```

```
# Read
df = read_sqlserver(CONN, "SELECT * FROM dbo.orders")

# Execute DDL / DML
execute_sqlserver(CONN, "DROP TABLE IF EXISTS dbo.orders")
execute_sqlserver(CONN, (
    "CREATE TABLE dbo.orders ("
    "id BIGINT, amount FLOAT, category NVARCHAR(MAX))"
))

# Write — append (batch INSERT 500 rows/statement)
rows = df.to_sqlserver(CONN, "dbo.orders", mode="append")

# Write — upsert (MERGE … WHEN MATCHED … WHEN NOT MATCHED …)
rows = df.to_sqlserver(CONN, "dbo.orders", mode="upsert", conflict_cols=["id"])
```

Node.js

```
const { read_sqlserver, execute_sqlserver } = require('./pardox/src/io');

const CONN = 'Server=localhost,1433;Database=mydb;UID=sa;PWD=MyPassword;TrustServerCertificate=Yes';

// Read
const df = read_sqlserver(CONN, 'SELECT * FROM dbo.orders');

// Execute DDL / DML
execute_sqlserver(CONN, 'DROP TABLE IF EXISTS dbo.orders');

// Write — append
const rows = df.toSqlserver(CONN, 'dbo.orders', 'append');

// Write — upsert (MERGE)
const rows = df.toSqlserver(CONN, 'dbo.orders', 'upsert', ['id']);
```

PHP

```
use PardoX\DataFrame;
use PardoX\IO;
```

```
$CONN = 'Server=localhost,1433;Database=mydb;UID=sa;PWD=MyPassword;TrustServerCertificate=Yes';

// Read
$df = IO::read_sqlserver($CONN, 'SELECT * FROM dbo.orders');

// Execute DDL / DML
IO::executeSqlserver($CONN, 'DROP TABLE IF EXISTS dbo.orders');

// Write — append
$rows = $df->to_sqlserver($CONN, 'dbo.orders', 'append');

// Write — upsert (MERGE)
$rows = $df->to_sqlserver($CONN, 'dbo.orders', 'upsert', ['id']);
```

Write Modes — SQL Server

Mode	Behavior
append	INSERT INTO table (cols) VALUES (r1),(r2),…,(r500) — 500 rows per statement.
replace	Same multi-row INSERT, replacing existing rows.
upsert	MERGE INTO table AS T USING (VALUES …) AS S ON (…) WHEN MATCHED … WHEN NOT MATCHED … — 500 rows per MERGE statement. Requires conflict_cols.

MongoDB

Python

```
from pardox.io import read_mongodb, execute_mongodb
import pardox

CONN = "mongodb://admin:secret@localhost:27017"

# Read — collection as "database.collection"
df = read_mongodb(CONN, "mydb.orders")

# Execute a MongoDB command
execute_mongodb(CONN, "mydb", '{"drop": "orders"}')

# Write — append
rows = df.to_mongodb(CONN, "mydb.orders", mode="append")

# Write — replace (drops collection, then inserts)
rows = df.to_mongodb(CONN, "mydb.orders", mode="replace")
```

Node.js

```
const { read_mongodb, execute_mongodb } = require('./pardox/src/io');

const CONN = 'mongodb://admin:secret@localhost:27017';

// Read
const df = read_mongodb(CONN, 'mydb.orders');

// Execute command
execute_mongodb(CONN, 'mydb', '{"drop": "orders"}');

// Write — append
const rows = df.toMongodb(CONN, 'mydb.orders', 'append');

// Write — replace
const rows = df.toMongodb(CONN, 'mydb.orders', 'replace');
```

PHP

```
use PardoX\DataFrame;
use PardoX\IO;

$CONN = 'mongodb://admin:secret@localhost:27017';

// Read
$df = IO::read_mongodb($CONN, 'mydb.orders');

// Execute command
IO::executeMongodb($CONN, 'mydb', '{"drop": "orders"}');

// Write — append
$rows = $df->to_mongodb($CONN, 'mydb.orders', 'append');

// Write — replace
$rows = $df->to_mongodb($CONN, 'mydb.orders', 'replace');
```

Write Modes — MongoDB

Mode	Behavior
append	insert_many in batches of 10,000 documents, ordered: false (continues on individual document errors).
replace	Drops the target collection, then inserts all documents in batches of 10,000.

API Reference Summary

Python (pardox.io)

Function
Returns
Description

read_sql(conn, query)
DataFrame
Read from PostgreSQL

execute_sql(conn, query)
int
DDL/DML on PostgreSQL

read_mysql(conn, query)
DataFrame
Read from MySQL

execute_mysql(conn, query)
int
DDL/DML on MySQL

read_sqlserver(conn, query)
DataFrame
Read from SQL Server

execute_sqlserver(conn, query)
int
DDL/DML on SQL Server

read_mongodb(conn, db.col)

DataFrame
Read a MongoDB collection

execute_mongodb(conn, db, cmd_json)
int
Run a MongoDB command

DataFrame write methods — Python

Method
Returns
Description

df.to_sql(conn, table, mode, conflict_cols)
int
Write to PostgreSQL

df.to_mysql(conn, table, mode, conflict_cols)
int
Write to MySQL

df.to_sqlserver(conn, table, mode, conflict_cols)
int
Write to SQL Server

df.to_mongodb(conn, db.col, mode)
int
Write to MongoDB

Node.js (pardox/src/io.js)

Function
Returns
Description

read_sql(conn, query)
DataFrame
Read from PostgreSQL

executeSql(conn, query)
number
DDL/DML on PostgreSQL

read_mysql(conn, query)
DataFrame
Read from MySQL

execute_mysql(conn, query)
number
DDL/DML on MySQL

read_sqlserver(conn, query)
DataFrame
Read from SQL Server

execute_sqlserver(conn, query)
number
DDL/DML on SQL Server

read_mongodb(conn, db.col)
DataFrame
Read a MongoDB collection

execute_mongodb(conn, db, cmd_json)
number
Run a MongoDB command

DataFrame write methods — Node.js

Method
Returns
Description

df.toSql(conn, table, mode, conflictCols)
number
Write to PostgreSQL

df.toMysql(conn, table, mode, conflictCols)
number
Write to MySQL

df.toSqlserver(conn, table, mode, conflictCols)
number
Write to SQL Server

df.toMongodb(conn, db.col, mode)
number
Write to MongoDB

PHP (PardoX\IO + PardoX\DataFrame)

Function
Returns
Description

IO::read_sql($conn, $query)
DataFrame
Read from PostgreSQL

IO::executeSql($conn, $query)
int
DDL/DML on PostgreSQL

IO::read_mysql($conn, $query)
DataFrame
Read from MySQL

IO::executeMysql($conn, $query)
int
DDL/DML on MySQL

IO::read_sqlserver($conn, $query)
DataFrame
Read from SQL Server

IO::executeSqlserver($conn, $query)
int

DDL/DML on SQL Server

IO::read_mongodb($conn, $db_col)
DataFrame
Read a MongoDB collection

IO::executeMongodb($conn, $db, $cmd_json)
int
Run a MongoDB command

DataFrame write methods — PHP

Method
Returns
Description

$df->to_sql($conn, $table, $mode, $conflictCols)
int
Write to PostgreSQL

$df->to_mysql($conn, $table, $mode, $conflictCols)
int
Write to MySQL

$df->to_sqlserver($conn, $table, $mode, $conflictCols)
int
Write to SQL Server

$df->to_mongodb($conn, $db_col, $mode)

int
Write to MongoDB

Return Values
All write methods return the number of rows written as an integer. All execute methods return the number of rows affected (0 for DDL statements like CREATE TABLE or DROP TABLE). On error, all methods throw an exception (Python: RuntimeError, Node.js: Error, PHP: RuntimeException).

Bulk Write Performance Summary

Database
Rows < 10,000
Rows ≥ 10,000 (append)
Mechanism

PostgreSQL
Multi-row INSERT
COPY FROM STDIN (auto)
Wire-level streaming, no temp files

MySQL
1,000 rows/stmt
LOAD DATA LOCAL INFILE (auto, if server allows) → fallback to 1,000 rows/stmt
In-memory CSV buffer

SQL Server
500 rows/stmt
500 rows/stmt
Multi-row INSERT … VALUES

MongoDB
10,000 docs/batch
10,000 docs/batch
insert_many(ordered: false)

Known Limitations

SQL Server — Special Characters in Password

Passwords containing ! (and potentially other special characters) cause the tiberius driver to send an incorrect authentication hash, resulting in a Login failed (State 8) error. This affects all SDKs since the issue is in the Rust core.

Workaround: Use a SQL Server password that contains only alphanumeric characters and _, -, @, #, $.

```
# Works
Server=localhost,1433;UID=sa;PWD=MyPassword_123;TrustServerCertificate=Yes

# Fails (tiberius bug with !)
Server=localhost,1433;UID=sa;PWD=MyP@ssw0rd!;TrustServerCertificate=Yes
```

A fix is tracked for the next release — see validation_complete/suggestions.md.

MySQL — LOAD DATA LOCAL INFILE Requires Server Configuration

The fast bulk path (LOAD DATA LOCAL INFILE) requires local_infile=ON on the MySQL server. If it is disabled (ERROR 3948), PardoX automatically falls back to chunked batch INSERT with no action required from the user.

To enable on the server side:

```
SET GLOBAL local_infile = 1;
```

Or in my.cnf:

```
[mysqld]
local_infile=1
```

Table Must Exist Before Writing

PardoX write methods (to_sql, to_mysql, etc.) append to an existing table. Create the table first using the corresponding execute_* function before calling a write method.

```
execute_sql(CONN, "CREATE TABLE IF NOT EXISTS orders (id BIGINT, amount FLOAT)")
```

rows = df.to_sql(CONN, "orders", mode="append")

Validated Versions (v0.3.1)

Database
Read
Write (50k rows)
SDK

PostgreSQL

COPY (0.x s)
Python, Node.js

MySQL

chunked batch
Python, PHP

SQL Server

⚠ password ! bug
—

MongoDB

not benchmarked
—

FFI Exports Reference

PardoX Core FFI Exports — v0.3.2
Complete reference for all C-ABI functions exported by the PardoX Rust core. Use this to build custom bindings, audit SDK integrations, or test functions directly.
Total exports: 181 functions across 5 crates
Last updated: 2026-03-15

Table of Contents

pardox_cpu — Main CPU Engine (161 functions)
pardox_gpu — GPU Acceleration (20 functions)
pardox_napi — Node.js Bindings (60+ functions)
pardox_wasm — WebAssembly (stub)
pardox_server — Server Component (stub)
Architectural Patterns

pardox_cpu
Path: Pardox-Core-v0.3.2/pardox_cpu/
Crate type: cdylib
Output: pardox-cpu-Linux-x64.so / pardox-cpu-Darwin-x64.dylib / pardox-cpu-Win-x64.dll
Engine Init (1)

Function
Signature
Description

pardox_init_engine
() -> void
Initialize engine, MiMalloc allocator, attempt GPU plugin load

api.rs — Core Lifecycle (6)

Function
Parameters
Return
Description

pardox_reset
()
void
Clear global HyperBlockManager state

pardox_create_block
(col_names_json: *const c_char, col_types_json: *const c_char)
*mut HyperBlock
Allocate a new HyperBlock

pardox_append_block
(block_ptr: *mut HyperBlock)
i64
Append HyperBlock to global manager

pardox_free_manager
(mgr_ptr: *mut HyperBlockManager)
void
UNSAFE — release Rust-allocated manager

pardox_hash_join
(left_mgr: *mut HyperBlockManager, right_mgr: *mut HyperBlockManager, left_key: *const c_char, right_key: *const c_char)
*mut HyperBlockManager
Hash join two managers on key columns

pardox_get_system_report
()
*const c_char
JSON system info (CPU, memory, version)

api_core.rs — Mutation & Transformation (3)

Function
Parameters
Return
Description

pardox_add_column
(target_mgr: *mut HyperBlockManager, source_mgr: *mut HyperBlockManager, name: *const c_char)
i64
Add column from source manager into target

pardox_fill_na
(mgr: *mut HyperBlockManager, col_name: *const c_char, value: *const c_char)
i64
Fill null values with given value

pardox_round
(mgr: *mut HyperBlockManager, col_name: *const c_char, decimals: i32)
i64
Round float column to N decimal places

api_reader.rs — Data Ingestion (13)

Function	Parameters	Return	Description
pardox_load_manager_csv	(path: *const c_char, schema_json: *const c_char, config_json: *const c_char)	*mut HyperBlockManager	Load CSV file (3 args — all required)
pardox_scan_csv	(path: *const c_char, schema_json: *const c_char, config_json: *const c_char)	i64	Scan CSV into global manager
pardox_read_json_bytes	(bytes_ptr: *const u8, len: usize)	*mut HyperBlockManager	Ingest JSON from raw bytes
pardox_load_manager_prdx	(path: *const c_char, limit: i64)	*mut HyperBlockManager	Load .prdx binary file with row limit
pardox_load_manager_parquet	(path: *const c_char)	*mut HyperBlockManager	Load Parquet file

pardox_read_head_json
(path: *const c_char, limit: i64)
*const c_char
Read first N rows of .prdx as JSON string

pardox_column_sum
(path: *const c_char, col: *const c_char)
f64
Sum a column directly from .prdx file

pardox_query_sql
(path: *const c_char, query: *const c_char)
*const c_char
Run SQL query on .prdx file, return JSON

pardox_prdx_min
(path: *const c_char, col: *const c_char)
f64
Min value directly from .prdx file

pardox_prdx_max
(path: *const c_char, col: *const c_char)
f64
Max value directly from .prdx file

pardox_prdx_mean
(path: *const c_char, col: *const c_char)
f64
Mean value directly from .prdx file

pardox_prdx_count
(path: *const c_char)
f64

Row count directly from .prdx file

pardox_groupby_agg_prdx
(path: *const c_char, group_cols_json: *const c_char, agg_json: *const c_char)
*mut HyperBlockManager
GroupBy aggregation directly from .prdx file

api_writers.rs — Export & Persistence (21)

Function
Parameters
Return
Description

pardox_to_csv
(mgr: *mut HyperBlockManager, path: *const c_char)
i64
Write manager to CSV file

pardox_to_prdx
(mgr: *mut HyperBlockManager, path: *const c_char)
i64
Write manager to .prdx binary format

pardox_write_parquet
(path: *const c_char)
i64
Write global manager to Parquet

pardox_write_sharded_parquet
(mgr: *mut HyperBlockManager, directory: *const c_char, shards: i32)
i64
Shard-write to Parquet files

pardox_hyper_copy_v3
(src_pattern: *const c_char, out_path: *const c_char, schema_json: *const c_char, config_json: *const c_char)
i64
Convert CSV glob pattern → .prdx (bulk ingest)

pardox_version
()
*const c_char
Return library version string

pardox_read_schema
(path: *const c_char)
*const c_char
Read schema from .prdx file as JSON

pardox_read_head
(path: *const c_char, rows: i64)
*const c_char
Read first N rows as JSON

pardox_legacy_ping
(system_name: *const c_char)
i64
Backward compat ping

pardox_import_mainframe_dat
(path: *const c_char, schema_json: *const c_char)
*mut HyperBlockManager
Import mainframe fixed-width .dat files

pardox_free_string
(ptr: *mut c_char)
void
Free Rust-allocated string pointer

pardox_write_sql
(mgr: *mut HyperBlockManager, conn_str: *const c_char, table_name: *const c_char, mode: *const c_char, conflict_cols_json: *const c_char)
i64
Write HyperBlockManager to PostgreSQL (COPY for >10k rows, INSERT otherwise)

pardox_write_sql_prdx
(prdx_path: *const c_char, conn_str: *const c_char, table_name: *const c_char, mode: *const c_char, conflict_cols_json: *const c_char, batch_rows: i64)
i64
Stream .prdx file directly to PostgreSQL via COPY FROM STDIN — O(block) RAM. Added v0.3.2. Error codes: -1=invalid path, -2=invalid conn, -3=invalid table, -4=invalid mode, -5=invalid JSON, -10=file not found, -20=empty conn, -100=engine error

pardox_write_mysql
(mgr: *mut HyperBlockManager, conn_str: *const c_char, table_name: *const c_char)
i64
Write to MySQL

pardox_write_sqlserver
(mgr: *mut HyperBlockManager, conn_str: *const c_char, table_name: *const c_char)
i64
Write to SQL Server

pardox_write_mongodb
(mgr: *mut HyperBlockManager, conn_str: *const c_char, db_collection: *const c_char)
i64
Write to MongoDB

pardox_sql_query
(mgr: *mut HyperBlockManager, sql: *const c_char)
*mut HyperBlockManager
Run SQL on in-memory manager

pardox_execute_sql
(conn_str: *const c_char, query: *const c_char)
*mut HyperBlockManager
Execute SQL on PostgreSQL, return results

pardox_execute_mysql
(conn_str: *const c_char, query: *const c_char)
*mut HyperBlockManager
Execute SQL on MySQL, return results

pardox_execute_sqlserver
(conn_str: *const c_char, query: *const c_char)
*mut HyperBlockManager
Execute SQL on SQL Server, return results

pardox_execute_mongodb
(conn_str: *const c_char, db_collection: *const c_char, query_json: *const c_char)
*mut HyperBlockManager
Execute query on MongoDB

api_ext.rs — Advanced Operations (118+)
Arrow & Memory

Function
Parameters
Return

pardox_ingest_arrow_stream
(array_ptr: *const u8, schema_ptr: *const u8)
*mut HyperBlockManager

pardox_ingest_buffer
(buffer_ptr: *const u8, buffer_len: usize, layout_json: *const c_char, schema_json: *const c_char)
*mut HyperBlockManager

Inspection & Serialization

Function
Parameters
Return

pardox_manager_to_json
(mgr: *mut HyperBlockManager, limit: i64)
*const c_char

pardox_manager_to_ascii
(mgr: *mut HyperBlockManager, limit: i64)
*const c_char

pardox_manager_to_json_range
(mgr: *mut HyperBlockManager, start: i64, limit: i64)
*const c_char

pardox_get_row_count
(mgr: *mut HyperBlockManager)
i64

pardox_get_schema_json
(mgr: *mut HyperBlockManager)
*const c_char

SQL Database Readers

Function
Parameters
Return

pardox_scan_sql
(conn_str: *const c_char, query: *const c_char)
*mut HyperBlockManager

pardox_read_mysql
(conn_str: *const c_char, query: *const c_char)
*mut HyperBlockManager

pardox_read_sqlserver
(conn_str: *const c_char, query: *const c_char)
*mut HyperBlockManager

pardox_read_mongodb
(conn_str: *const c_char, db_dot_collection: *const c_char)
*mut HyperBlockManager

JSON Export

Function
Parameters
Return

pardox_to_json_records
(mgr: *mut HyperBlockManager)
*const c_char

pardox_to_json_arrays
(mgr: *mut HyperBlockManager)
*const c_char

EDA

Function
Parameters
Return

pardox_value_counts
(mgr: *mut HyperBlockManager, col_name: *const c_char)
*const c_char

pardox_unique
(mgr: *mut HyperBlockManager, col_name: *const c_char)
*const c_char

Slicing

Function
Parameters
Return

pardox_slice_manager
(mgr: *mut HyperBlockManager, start: i64, len: i64)
*mut HyperBlockManager

pardox_tail_manager
(mgr: *mut HyperBlockManager, n: i64)
*mut HyperBlockManager

Type Casting

Function
Parameters
Return

pardox_cast_column
(mgr: *mut HyperBlockManager, col_name: *const c_char, target_type_str: *const c_char)

i32

Series Arithmetic

Function
Parameters
Return

pardox_series_add
(left_mgr, left_col, right_mgr, right_col)
*mut HyperBlockManager

pardox_series_sub
same pattern
*mut HyperBlockManager

pardox_series_mul
same pattern
*mut HyperBlockManager

pardox_series_div
same pattern
*mut HyperBlockManager

pardox_series_mod
same pattern
*mut HyperBlockManager

Filtering & Comparison

Function
Parameters
Return

pardox_filter_compare
(left_mgr, left_col, right_mgr, right_col, op_code: i32)
*mut HyperBlockManager

pardox_filter_compare_scalar
(mgr, col_name, val_f64: f64, val_i64: i64, is_float: i32, op_code: i32)
*mut HyperBlockManager

pardox_apply_filter
(data_mgr, mask_mgr, mask_col)
*mut HyperBlockManager

Aggregations

Function
Parameters
Return

pardox_agg_sum
(mgr, col_name)
f64

pardox_agg_mean
(mgr, col_name)
f64

pardox_agg_min
(mgr, col_name)
f64

pardox_agg_max
(mgr, col_name)
f64

pardox_agg_count
(mgr, col_name)
f64

pardox_agg_std
(mgr, col_name)
f64

Math

Function
Parameters
Return

pardox_math_add
(mgr, col_a, col_b)

*mut HyperBlockManager

pardox_math_sub
(mgr, col_a, col_b)
*mut HyperBlockManager

pardox_math_stddev
(mgr, col_name)
f64

pardox_math_minmax
(mgr, col_name)
*mut HyperBlockManager

Sorting

Function
Parameters
Return

pardox_sort_values
(mgr, col_name, descending: i32)
*mut HyperBlockManager

pardox_gpu_sort
(mgr, col_name)
*mut HyperBlockManager

Buffer Access (NumPy / ML integration)

Function
Parameters
Return

pardox_get_f64_buffer
(mgr, col_name, out_len: *mut usize)
*const f64

GroupBy

Function
Parameters
Return

pardox_groupby_agg
(mgr, group_cols_json, agg_json)
*mut HyperBlockManager

Gap-specific Functions
Gap 2 — String & Date

Function

Parameters

Return

pardox_str_upper

(mgr, col_name)

*mut HyperBlockManager

pardox_str_lower

(mgr, col_name)

*mut HyperBlockManager

pardox_str_contains

(mgr, col_name, pattern)

*mut HyperBlockManager

pardox_str_replace

(mgr, col_name, old_str, new_str)

*mut HyperBlockManager

pardox_str_trim

(mgr, col_name)

*mut HyperBlockManager

pardox_str_len

(mgr, col_name)

*mut HyperBlockManager

pardox_date_extract

(mgr, col_name, part: *const c_char)

*mut HyperBlockManager

pardox_date_format
(mgr, col_name, format)
*mut HyperBlockManager

pardox_date_diff
(mgr, col1, col2, unit)
*mut HyperBlockManager

pardox_date_add
(mgr, col_name, days: i64)
*mut HyperBlockManager

Gap 3 — Decimal Type

Function
Parameters
Return

pardox_decimal_from_float
(mgr, col_name, scale: i32)
*mut HyperBlockManager

pardox_decimal_sum
(mgr, col_name)
f64

pardox_decimal_get_scale
(mgr, col_name)
i32

pardox_decimal_add
(mgr, col_name, val: f64)
*mut HyperBlockManager

pardox_decimal_sub
(mgr, col_name, val: f64)
*mut HyperBlockManager

pardox_decimal_mul_float
(mgr, col_name, factor: f64)
*mut HyperBlockManager

pardox_decimal_round
(mgr, col_name, scale: i32)
*mut HyperBlockManager

pardox_decimal_to_float
(mgr, col_name)
*mut HyperBlockManager

Gap 4 — Window Functions

Function
Parameters
Return

pardox_row_number
(mgr, order_col)
*mut HyperBlockManager

pardox_rank
(mgr, order_col)
*mut HyperBlockManager

pardox_lag
(mgr, col_name, offset: i64)
*mut HyperBlockManager

pardox_lead
(mgr, col_name, offset: i64)
*mut HyperBlockManager

pardox_rolling_mean
(mgr, col_name, window_size: i64)
*mut HyperBlockManager

Gap 5 — Lazy Pipeline

Function
Parameters
Return

pardox_lazy_scan_csv
(path, schema_json, config_json)
*mut LazyFrame

pardox_lazy_scan_prdx
(path)

*mut LazyFrame

pardox_lazy_select
(lazy_ptr, cols_json)
*mut LazyFrame

pardox_lazy_filter
(lazy_ptr, col_name, op, value: f64)
*mut LazyFrame

pardox_lazy_limit
(lazy_ptr, n: i64)
*mut LazyFrame

pardox_lazy_collect
(lazy_ptr)
*mut HyperBlockManager

pardox_lazy_describe
(lazy_ptr)
*const c_char

pardox_lazy_optimize
(lazy_ptr)
*mut LazyFrame

pardox_lazy_stats
(lazy_ptr)
*const c_char

pardox_lazy_free
(lazy_ptr)
void

Gap 8 — Pivot & Melt

Function
Parameters
Return

pardox_pivot_table
(mgr, index_col, columns_col, values_col, agg_func)
*mut HyperBlockManager

pardox_melt
(mgr, id_vars_json, value_vars_json)
*mut HyperBlockManager

Gap 9 — Time Series Fill

Function
Parameters
Return

pardox_ffill
(mgr, col_name)
*mut HyperBlockManager

pardox_bfill
(mgr, col_name)
*mut HyperBlockManager

pardox_interpolate
(mgr, col_name, method)
*mut HyperBlockManager

Gap 10 — Nested Data

Function
Parameters
Return

pardox_json_extract
(mgr, col_name, json_path)
*mut HyperBlockManager

pardox_explode
(mgr, col_name)
*mut HyperBlockManager

pardox_unnest
(mgr, col_name)
*mut HyperBlockManager

Gap 11 — Spill to Disk

Function
Parameters
Return

pardox_spill_to_disk
(mgr, path)
i64

pardox_spill_from_disk
(path)
*mut HyperBlockManager

pardox_memory_usage
(mgr)
i64

Gap 14 — External Sort / Streaming GroupBy

Function
Parameters
Return

pardox_external_sort
(mgr, col_name)
*mut HyperBlockManager

pardox_chunked_groupby

(mgr, group_cols_json, chunk_size: i64)
*mut HyperBlockManager

Gap 15 — Cloud Storage

Function
Parameters
Return

pardox_cloud_read_csv
(url, config_json)
*mut HyperBlockManager

pardox_cloud_read_prdx
(url)
*mut HyperBlockManager

pardox_cloud_write_prdx
(mgr, url, config_json)
i64

Gap 16 — Live Query

Function
Parameters
Return

pardox_live_query_start
(mgr, poll_interval_ms: i64)
*mut LiveQuery

pardox_live_query_version
(lq_ptr)
i64

pardox_live_query_take
(lq_ptr, n_rows: i64)
*mut HyperBlockManager

pardox_live_query_free
(lq_ptr)
void

Gap 18 — Encryption

Function
Parameters
Return

pardox_write_prdx_encrypted
(mgr, path, password)
i64

pardox_read_prdx_encrypted
(path, password)

*mut HyperBlockManager

Gap 19 — Data Contracts

Function
Parameters
Return

pardox_validate_contract
(mgr, schema_json)
i64

pardox_contract_violation_count
()
i64

Gap 20 — Time Travel

Function
Parameters
Return

pardox_version_write
(mgr, path, version_id)
i64

pardox_version_read
(path, version_id)
*mut HyperBlockManager

pardox_version_list
(path)
*const c_char

pardox_version_delete
(path, version_id)
i64

Gap 21 — Arrow Flight

Function
Parameters
Return

pardox_flight_start
(port: i32)
i64

pardox_flight_register
(path, mgr)
i64

pardox_flight_stop
()
i32

pardox_flight_read
(table_ref)
*mut HyperBlockManager

Gap 22 — Distributed Cluster

Function
Parameters
Return

pardox_cluster_worker_start
(listen_addr, port: i32)
i64

pardox_cluster_worker_stop
()
i32

pardox_cluster_worker_register
(name, listen_addr)
i64

pardox_cluster_connect
(worker_addresses_json)
i64

pardox_cluster_scatter
(mgr, partitions: i32)

*const c_char

pardox_cluster_scatter_resilient
(mgr, partitions: i32, replication_factor: i32)
*const c_char

pardox_cluster_sql
(sql_query)
*mut HyperBlockManager

pardox_cluster_sql_resilient
(sql_query)
*mut HyperBlockManager

pardox_cluster_ping
(worker_name)
i64

pardox_cluster_ping_all
()
i64

pardox_cluster_checkpoint
(path)
i64

pardox_cluster_free
()
void

Gap 23 — SQL Server Fix

Function
Parameters
Return

pardox_sqlserver_config_ok
()
i32

Gap 28 — Linear Algebra

Function
Parameters
Return

pardox_matmul
(left_mgr, left_col, right_mgr, right_col)
*mut HyperBlockManager

pardox_l2_normalize
(mgr, col_name)
*mut HyperBlockManager

pardox_l1_normalize
(mgr, col_name)
*mut HyperBlockManager

pardox_cosine_sim
(left_mgr, left_col, right_mgr, right_col)
f64

pardox_pca
(mgr, n_components: i32)
*mut HyperBlockManager

Gap 29 — REST Connector

Function
Parameters
Return

pardox_read_rest
(url, method, headers_json)
*mut HyperBlockManager

pardox_gpu
Path: Pardox-Core-v0.3.2/pardox_gpu/
Crate type: cdylib
Output: pardox-gpu-Linux-x64.so
Backend: wgpu (WebGPU — cross-platform: Vulkan, Metal, DX12, OpenGL ES)
Init

Function
Parameters
Return

Description

gpu_init
()
i64
Initialize wgpu device; 0 = success, -1 = no adapter

Binary Arithmetic (element-wise, f64)

Function
Parameters
Return

gpu_add_f64
(ptr_a: *const f64, ptr_b: *const f64, len: usize, ptr_out: *mut f64)
i64

gpu_sub_f64
same pattern
i64

gpu_mul_f64
same pattern
i64

gpu_div_f64
same pattern
i64

gpu_mod_f64
same pattern
i64

gpu_pow_f64
same pattern
i64

gpu_max_elementwise_f64
same pattern
i64

gpu_min_elementwise_f64
same pattern
i64

Reductions

Function
Parameters
Return

gpu_sum_f64
(ptr_in: *const f64, len: usize, ptr_out: *mut f64)
i64

gpu_min_f64
same pattern

i64

gpu_max_f64
same pattern
i64

gpu_reduce_sum_f64
same pattern (multi-pass)
i64

gpu_reduce_min_f64
same pattern
i64

gpu_reduce_max_f64
same pattern
i64

Sorting

Function
Parameters
Return
Description

gpu_sort_f64
(ptr_in: *const f64, len: usize, ptr_out: *mut f64, descending: i32)
i64
Bitonic sort on GPU

Unary Math

Function
Parameters
Return

gpu_sqrt_f64
(ptr_in: *const f64, len: usize, ptr_out: *mut f64)
i64

gpu_exp_f64
same pattern
i64

gpu_log_f64
same pattern
i64

gpu_abs_f64
same pattern
i64

pardox_napi
Path: Pardox-Core-v0.3.2/pardox_napi/
Crate type: cdylib
Output: pardox.node (Node.js native addon)
Binding framework: napi-rs
DataFrame class (30+ instance methods)

Method
Description

fromCsv(path, schema, config)
Load CSV

fromPrdx(path, limit)
Load .prdx

toJson(limit)
Serialize to JSON records

toAscii(limit)
Pretty-print table

groupBy(groupCols, aggJson)
GroupBy aggregation

sortValues(col, desc)
Sort

filter(col, op, value)
Scalar filter

select(cols)
Column selection

head(n) / tail(n)
Row slicing

getSchema()
Schema as JSON

rowCount()
Row count

sum/mean/min/max/std(col)
Aggregations

toCsv(path) / toPrdx(path)
Export

strUpper/Lower/Contains/Replace/Trim(col, ...)
String ops

dateExtract/Format/Diff/Add(col, ...)
Date ops

lag/lead/rank/rowNumber/rollingMean(col, ...)
Window ops

pivot/melt(...)
Reshape

ffill/bfill/interpolate(col, ...)
Time series fill

jsonExtract/explode/unnest(col, ...)

Nested data

spillToDisk/spillFromDisk(path)
Spill

encryptedWrite/encryptedRead(path, key)
Encryption

validateContract(schemaJson)
Contracts

versionWrite/Read/List/Delete(...)
Time travel

l2Normalize/l1Normalize/cosineSim/pca/matmul(...)
LinAlg

free()
Release memory

All pardox_cpu functions are also exposed as module-level exports with camelCase naming (60+ total).

pardox_wasm
Path: Pardox-Core-v0.3.2/pardox_wasm/
Target: wasm32-unknown-unknown
Binding framework: wasm-bindgen
Status: Stub / development placeholder
Exports a minimal PardoXWasm class with placeholder methods. Full WASM port planned.

pardox_server
Path: Pardox-Core-v0.3.2/pardox_server/
Status: Empty scaffold — no exported functions

The PostgreSQL wire-protocol server is implemented in Python (pardox_validation/py_validation/pardox/server/), not in this Rust crate.

Architectural Patterns

Memory Ownership

All *mut HyperBlockManager pointers are Rust-owned — caller must call pardox_free_manager(ptr) when done
String returns (*const c_char) are also Rust-owned — call pardox_free_string(ptr) to release
LazyFrame* pointers are consumed by pardox_lazy_collect() — do not free separately after collect

Error Signaling

i64 returns: >= 0 = success, < 0 = error code
*mut T returns: NULL = error
f64 returns: NaN or negative sentinel for aggregation errors

FFI Calling Conventions

All functions: extern "C" with #[no_mangle]
All strings: null-terminated UTF-8 C strings (*const c_char)
Boolean flags: i32 (0 = false, 1 = true)

Key FFI Gotchas

pardox_load_manager_csv requires exactly 3 args: (path, schema_json, config_json) — both JSON args can be "{}" but must not be NULL
pardox_get_f64_buffer takes out_len: *mut usize — caller provides pointer to receive length
pardox_lazy_filter arg order: (lazy_ptr, col_name, op_str, value: f64) — op is lowercase ("gt", "lt", "eq", "neq", "gte", "lte")
pardox_lazy_select expects JSON array string: '["col1","col2"]'
Decimal type code in schema JSON: 0x80 | scale (e.g., scale=2 → type code 130)
pardox_hyper_copy_v3 is for CSV→PRDX bulk conversion, NOT PostgreSQL COPY protocol

Platform Binaries

Platform
CPU

GPU
NAPI

Linux x64
pardox-cpu-Linux-x64.so
pardox-gpu-Linux-x64.so
pardox-Linux-x64.node

macOS Intel
pardox-cpu-Darwin-x64.dylib
pardox-gpu-Darwin-x64.dylib
pardox-Darwin-x64.node

macOS ARM
pardox-cpu-Darwin-arm64.dylib
pardox-gpu-Darwin-arm64.dylib
pardox-Darwin-arm64.node

Windows
pardox-cpu-Win-x64.dll
pardox-gpu-Win-x64.dll
pardox-Win-x64.node

Full API Reference

API Reference

Complete documentation of all classes, functions, and methods in the PardoX Python SDK v0.3.2.

Top-Level Functions (import pardox as px)

read_csv

Reads a CSV file into a DataFrame using the multi-threaded Rust parser.

```
def read_csv(path: str, schema: dict | None = None) -> DataFrame
```

Parameter

Type

Description

path

str

Path to the .csv file.

schema

dict or None

Optional column type overrides: {"col": "Float64", ...}. Supported types: Int64, Float64, Utf8.

Returns: DataFrame

```
df = px.read_csv("sales.csv")
df = px.read_csv("sales.csv", schema={"price": "Float64", "id": "Int64"})
```

read_prdx

Loads a native PardoX binary file (.prdx).

def read_prdx(path: str) -> list[dict]

Parameter
Type
Description

path
str
Path to the .prdx file.

Returns: list[dict] (preview rows)

from_arrow
Zero-copy conversion from a PyArrow Table or RecordBatch.
def from_arrow(data: pyarrow.Table | pyarrow.RecordBatch) -> DataFrame

```
import pyarrow as pa, pardox as px
arrow_table = pa.Table.from_pydict({"a": [1, 2, 3]})
df = px.from_arrow(arrow_table)
```

pardox.io — Database I/O
All database functions bypass the Python runtime — connection and data transfer happen entirely in the Rust core.
PostgreSQL
read_sql(connection_string, query) → DataFrame

```
from pardox.io import read_sql
df = read_sql("postgresql://user:pass@localhost:5432/db", "SELECT * FROM orders")
```

execute_sql(connection_string, query) → int
Executes DDL or DML. Returns rows affected (0 for DDL).

```
from pardox.io import execute_sql
execute_sql(CONN, "DROP TABLE IF EXISTS orders")
execute_sql(CONN, "CREATE TABLE orders (id BIGINT, amount FLOAT)")
n = execute_sql(CONN, "DELETE FROM orders WHERE status = 'cancelled'")
```

Raises: RuntimeError on connection or SQL failure.

MySQL

read_mysql(connection_string, query) → DataFrame

```
from pardox.io import read_mysql
df = read_mysql("mysql://user:pass@localhost:3306/db", "SELECT * FROM products")
```

execute_mysql(connection_string, query) → int

```
from pardox.io import execute_mysql
execute_mysql(CONN, "CREATE TABLE IF NOT EXISTS products (id BIGINT, price DOUBLE)")
```

SQL Server

read_sqlserver(connection_string, query) → DataFrame

```
from pardox.io import read_sqlserver
CONN = "Server=localhost,1433;Database=mydb;UID=sa;PWD=MyPwd;TrustServerCertificate=Yes"
df = read_sqlserver(CONN, "SELECT TOP 1000 * FROM dbo.orders")
```

execute_sqlserver(connection_string, query) → int

```
from pardox.io import execute_sqlserver
execute_sqlserver(CONN, "DROP TABLE IF EXISTS dbo.orders_bak")
```

Warning: Avoid ! in SQL Server passwords. Known tiberius v0.12 bug — fix planned for v0.3.2.

MongoDB

read_mongodb(connection_string, db_dot_collection) → DataFrame

```
from pardox.io import read_mongodb
df = read_mongodb("mongodb://admin:pass@localhost:27017", "mydb.orders")
```

execute_mongodb(connection_string, database, command_json) → int

```
from pardox.io import execute_mongodb
execute_mongodb("mongodb://...", "mydb", '{"drop": "orders_archive"}')
```

Class: DataFrame

The main data structure. Holds an opaque pointer to a Rust HyperBlockManager.

Construction

```
# From CSV
df = px.read_csv("file.csv")

# From SQL
df = read_sql(conn, "SELECT ...")

# From MySQL / SQL Server / MongoDB
df = read_mysql(conn, query)
df = read_sqlserver(conn, query)
df = read_mongodb(conn, "db.collection")

# From Arrow
df = px.from_arrow(arrow_table)
```

Properties

shape → tuple[int, int]

```
rows, cols = df.shape
print(f"{rows:,} rows × {cols} columns")
```

columns → list[str]

```
print(df.columns)   # ['id', 'price', 'quantity', ...]
```

dtypes → dict[str, str]

```
print(df.dtypes)    # {'id': 'Utf8', 'price': 'Float64', 'quantity': 'Int64'}
```

Inspection

show(n=10)

Prints the first n rows as an ASCII table to stdout.

```
df.show(5)
```

head(n=5) → DataFrame

Returns a new DataFrame with the first n rows.

```
top5 = df.head(5)
```

tail(n=5) → DataFrame

Returns a new DataFrame with the last n rows.

```
last5 = df.tail(5)
```

iloc(start, end) → DataFrame

Returns rows in the range [start, end).

```
subset = df.iloc(100, 200)   # rows 100–199
```

Type Operations

cast(col, target_type) → DataFrame

Converts a column to a new type in-place. Returns self.

```
df.cast("quantity", "Float64")
df.cast("id",       "Utf8")
```

Supported types: Int64, Float64, Utf8

Arithmetic Methods

All arithmetic methods return a new DataFrame with the result stored in a named column.

mul(col_a, col_b) → DataFrame

```
revenue_df = df.mul("price", "quantity")   # result column: 'result_mul'
```

add(col_a, col_b) → DataFrame

```
total_df = df.add("price", "tax")          # result column: 'result_add'
```

sub(col_a, col_b) → DataFrame

```
profit_df = df.sub("revenue", "cost")      # result column: 'result_sub'
```

std(col) → float

Sample standard deviation of a column. Pure Rust, no NumPy.

```
std_val = revenue_df.std("result_mul")
```

min_max_scale(col) → DataFrame

Normalizes column values to [0, 1]. Returns new DataFrame with result_minmax.

```
normed_df = df.min_max_scale("price")
```

Sorting

sort_values(by, ascending=True, gpu=False) → DataFrame

Sorts the DataFrame by a Float64 column. Returns a new sorted DataFrame.

```
sorted_df = df.sort_values("price", ascending=True)
sorted_df = df.sort_values("price", ascending=False, gpu=True)
```

Parameter

Type

Description

by

str

Column name to sort by. Must be Float64.

ascending

bool

True = ascending (default).

gpu
bool
Use GPU Bitonic sort. Falls back to CPU if GPU unavailable.

Filtering
filter(mask: Series) → DataFrame
Applies a boolean Series as a row filter. Returns a new DataFrame.

```
mask   = df['price'] > 100.0
result = df.filter(mask)
```

Data Cleaning
fillna(value: float) → DataFrame
Fills NaN / null values in all numeric columns in-place.

```
df.fillna(0.0)
```

round(decimals: int) → DataFrame
Rounds all numeric columns in-place.

```
df.round(2)
```

Observer — Export & Inspection
to_dict() → list[dict]
Returns all rows as a list of dictionaries (records format).

```
records = df.to_dict()
# [{'price': 19.99, 'state': 'TX', ...}, ...]
```

Returns: list[dict]
to_json() → str
Returns all rows as a JSON string "[{...}, ...]".

```
json_str = df.to_json()
```

Returns: str

value_counts(col) → dict[str, int]

Frequency of each unique value in a column, sorted by count descending.

```
state_dist = df.value_counts("state")
# {'TX': 6345, 'CA': 6301, ...}
```

Returns: dict[str, int]

unique(col) → list

Unique values in a column in insertion order.

```
cats = df.unique("category")
# ['Electronics', 'Books', ...]
```

Returns: list

Joins

join(other, on=None, left_on=None, right_on=None) → DataFrame

Hash-join two DataFrames on a key column.

```
result = orders.join(customers, on="customer_id")
result = orders.join(customers, left_on="cust_id", right_on="id")
```

Writers

to_prdx(path) → bool

Saves DataFrame to native binary format.

```
df.to_prdx("output.prdx")
```

to_csv(path) → bool

Exports DataFrame to a CSV file.

```
df.to_csv("output.csv")
```

to_sql(connection_string, table_name, mode="append", conflict_cols=[]) → int

Writes to PostgreSQL.

```
rows = df.to_sql(CONN, "orders", mode="append")
rows = df.to_sql(CONN, "orders", mode="upsert", conflict_cols=["id"])
```

Parameter
Type
Values

mode
str
"append", "upsert"

conflict_cols
list[str]
Columns for ON CONFLICT clause (upsert only)

Returns: int — rows written. Raises: RuntimeError on failure.

write_sql_prdx — PRDX Streaming to PostgreSQL
Added in v0.3.2
Stream a .prdx file directly to PostgreSQL via COPY FROM STDIN — O(block) RAM regardless of file size. The schema is read from the PRDX footer; data is never fully loaded into memory.

```
from pardox import write_sql_prdx

rows = write_sql_prdx(
    prdx_path,        # str — path to .prdx file
    connection_string, # str — PostgreSQL connection string
    table_name,        # str — target table (must already exist)
    mode="append",     # str — only "append" supported
    conflict_cols=[],  # list[str] — reserved for future upsert support
    batch_rows=1000000 # int — rows per COPY batch
)
print(f"Streamed {rows:,} rows")
```

Parameter
Type
Description

prdx_path
str
Path to the .prdx file

connection_string
str
PostgreSQL connection string (postgresql://user:pass@host:port/db)

table_name
str
Target table name (must exist with matching schema)

mode
str
Write mode — only "append" supported in v0.3.2

conflict_cols
list[str]
Reserved — pass []

batch_rows
int
Rows per COPY batch (default: 1,000,000)

Returns: int — total rows written. Raises: RuntimeError on failure.
Validated: 150M rows / 3.8 GB PRDX → PostgreSQL in ~490s at ~300,000 rows/s.
to_mysql(connection_string, table_name, mode="append", conflict_cols=[]) → int
Writes to MySQL.
rows = df.to_mysql(CONN, "products", mode="append")
rows = df.to_mysql(CONN, "products", mode="replace")
rows = df.to_mysql(CONN, "products", mode="upsert", conflict_cols=["id"])

Parameter
Type
Values

mode
str
"append", "replace", "upsert"

to_sqlserver(connection_string, table_name, mode="append", conflict_cols=[]) → int
Writes to SQL Server (batch INSERT 500 rows/stmt).
rows = df.to_sqlserver(CONN, "dbo.orders", mode="append")
rows = df.to_sqlserver(CONN, "dbo.orders", mode="upsert", conflict_cols=["id"])

Parameter
Type
Values

mode
str
"append", "replace", "upsert"

to_mongodb(connection_string, db_dot_collection, mode="append") → int
Writes to MongoDB (10,000 docs/batch, ordered: false).
rows = df.to_mongodb(CONN, "mydb.orders", mode="append")
rows = df.to_mongodb(CONN, "mydb.orders", mode="replace")

Parameter
Type
Values

mode
str
"append", "replace"

Class: Series
A single column view into a DataFrame. Returned by df['col_name']. Does not own the underlying memory — the parent DataFrame does.
Properties
name → str
Column name.
dtype → str
Column type ("Int64", "Float64", "Utf8").

Arithmetic Operators
Operations dispatch to SIMD-accelerated Rust kernels. All return a new Series.

```
total = df['price'] * df['quantity']
net   = df['total'] - df['discount']
tax   = df['total'] + df['tax_amount']
unit  = df['revenue'] / df['quantity']
```

Comparison Operators

Return a boolean Series usable as a filter mask.

Method	Meaning
s.eq(val)	==
s.neq(val)	!=
s.gt(val)	>
s.gte(val)	>=
s.lt(val)	<
s.lte(val)	<=

```
mask = df['price'].gt(100.0)
df_filtered = df.filter(mask)

mask2 = df['state'].eq("TX")
df_tx = df.filter(mask2)
```

Aggregations

All aggregation methods return a Python scalar.

Method	Returns	Description
sum()	float	Sum of all non-null values
mean()	float	Arithmetic mean
min()	float	Minimum value
max()	float	Maximum value

std()
float
Sample standard deviation

count()
int
Count of non-null values

```
total   = df['revenue'].sum()
average = df['revenue'].mean()
high    = df['revenue'].max()
low     = df['revenue'].min()
spread  = df['revenue'].std()
valid   = df['id'].count()
```

Transformations
fillna(value) → Series
df['price'].fillna(0.0)

round(decimals) → Series
df['price'].round(2)

NumPy Zero-Copy

```
import numpy as np

# Direct pointer into Rust buffer — no allocation
arr = np.array(df['price'])   # dtype: float64
```

Works on Float64 columns. Cast Int64 columns first:

```
df.cast("quantity", "Float64")
arr = np.array(df["quantity"])
```

Error Codes

All database functions raise RuntimeError with a descriptive message on failure. The underlying Rust function returns integer error codes:

Code	Meaning
-1	Invalid manager pointer (null)
-2	Invalid connection string
-3	Invalid table / query string
-4	Invalid mode string
-5	Invalid conflict columns JSON
-10	File not found (write_sql_prdx only)
-20	Empty connection string (write_sql_prdx only)

-100

Operation failed — check stderr for Rust error details

Tip: When -100 is returned, the Rust core logs the actual database error to stderr before returning. Run with stderr visible to diagnose connection or schema issues.

Base Knowledge — Validation Scripts Index

Base Knowledge

Validation scripts for PardoX v0.3.2 across all supported SDKs. Each file is a standalone, runnable example that verifies one or more features of the SDK.

SDKs

Folder
SDK
Install
Files

python_code/
Python 3.8+
pip install pardox
28

js_code/
Node.js 18+
npm i @pardox/pardox
18

php_code/
PHP 8.1+
composer require betoalien/pardox-php
28

Gaps covered

File pattern
Feature

validate_gap1_sdk.*
GroupBy Aggregation

validate_gap2_sdk.*
String & Date Operations

validate_gap3_sdk.*
Decimal Type

validate_gap4_sdk.*
Window Functions

validate_gap5_sdk.*
Lazy Pipeline

validate_gap6_sdk.*
PostgreSQL Wire Server

validate_gap7_sdk.*
GPU Sort

validate_gap8_sdk.*
Pivot & Melt

validate_gap9_sdk.*
Time Series Fill

validate_gap10_sdk.*
Nested / JSON Data

validate_gap11_sdk.*
Out-of-Core / Spill to Disk

validate_gap12_sdk.*
PRDX Loader

validate_gap13_sdk.*
Streaming GroupBy on .prdx

validate_gap14_sdk.*
SQL over DataFrames

validate_gap15_sdk.*
Cloud Storage (S3 / GCS / Azure)

validate_gap16_sdk.*
Live Query

validate_gap17_sdk.*
WebAssembly

validate_gap18_sdk.*
Encryption

validate_gap19_sdk.*
Data Contracts (Python, PHP only)

validate_gap20_sdk.*
Time Travel (Python, PHP only)

validate_gap21_sdk.*
Arrow Flight (Python, PHP only)

validate_gap22_sdk.*
Distributed Cluster (Python, PHP only)

validate_gap23_sdk.*
SQL Server (Python, PHP only)

validate_gap24_sdk.*
Hash JOIN (Python, PHP only)

validate_gap26_sdk.*
Fault Tolerance (Python, PHP only)

validate_gap27_sdk.*
Query Planner (Python, PHP only)

validate_gap28_sdk.*
Linear Algebra (Python, PHP only)

validate_gap29_sdk.*
REST Connector (Python, PHP only)

How to run
Python

```
python validate_gap1_sdk.py

# Node.js
node validate_gap1_sdk.js

# PHP
php validate_gap1_sdk.php
```

Each script prints a results summary. All tests should show ALL TESTS PASSED ✓.